COMPLETE STUDY NOTES · 18 CHAPTERS · BEGINNER TO ADVANCED

GCP Cloud & DevOps — Master Study Guide

A complete, single-source reference to Google Cloud Platform & DevOps — from cloud fundamentals through Compute Engine, load balancing, storage, VPC networking, Cloud Storage, Cloud IAM, Cloud SQL, serverless (App Engine/Cloud Functions), and Terraform. Every theory note, diagram description, command table, and scenario-based interview Q&A from the source material is preserved in full, organized by the document's own service categories.

Cloud Fundamentals · Compute Services · Storage Services · Networking · Object Storage · Identity & Access · Databases · Serverless · Infrastructure as Code

INDEX · JUMP TO A CHAPTER

OV

1. Introduction to Cloud Computing

Cloud computing models, CapEx vs OpEx, Shared Fate, and GCP costing.

Cloud Fundamentals

GCE

2. Compute Engine — Overview & Instance Launch

Compute Engine — images, machine types, and launching your first VM.

Compute Services

GCEOP

3. Compute Engine — Instance Management

Instance lifecycle — OS Login, firewall rules, disks, snapshots, metadata.

Compute Services

LB

4. Load Balancing

Distributing traffic — global and regional HTTP(S) and network load balancers.

Compute Services

MON

5. Billing and Monitoring

Cloud Billing and Cloud Monitoring — budgets, metrics, alerts, logging.

Compute Services

SCALE

6. Autoscaler

Managed Instance Groups and reactive/scheduled/queue-based autoscaling.

Compute Services

DISK

7. Storage Services — Persistent Disk & Cloud Filestore

Persistent Disk and Cloud Filestore — block storage vs shared NFS.

Storage Services

VPC

8. Networking — VPC Network

VPC Network fundamentals — a global resource, regional subnets, CIDR.

Networking

VPCC

9. Networking — VPC Controls

Firewall rules, routes, Cloud NAT, VPN/Interconnect, hub-and-spoke.

Networking

GCS

10. Object Storage — Cloud Storage

Cloud Storage fundamentals — buckets, storage classes, versioning, signed URLs.

Object Storage

GCSA

11. Object Storage — Cloud Storage Advanced

Partitioning, gcloud storage transfer, replication, lifecycle, KMS.

Object Storage

GCSX

12. Object Storage — Cross-Project Cloud Storage Access

Cross-project Cloud Storage access and VPC Service Controls.

Object Storage

IAM

13. Identity & Access — Cloud IAM

Cloud IAM — 2-Step Verification, roles, service accounts, auditing.

Identity & Access

SQL

14. Databases — Cloud SQL

Cloud SQL — PaaS vs IaaS, Cloud SQL Auth Proxy, Memorystore.

Databases

SQLOP

15. Databases — Cloud SQL Operations

Connecting to Cloud SQL, running queries, and read replicas.

Databases

APP

16. Serverless — Cloud Functions & App Engine

App Engine, Cloud Functions, and Cloud Run — serverless compute options.

Serverless

APPI

17. Serverless — Cloud Functions Integrations & Limitations

Function limitations and integrations — Cloud SQL, Firestore, API Gateway.

Serverless

TF

18. Infrastructure as Code — Terraform with GCP

Terraform with the google provider — state, modules, CI/CD workflow.

Infrastructure as Code

OV

CHAPTER 01 · CLOUD FUNDAMENTALS

Introduction to Cloud Computing

Cloud computing models, CapEx vs OpEx, Shared Fate, and GCP costing.

Cloud Computing is the on-demand delivery of IT resources — compute, storage, databases, networking, and software — over the internet, with pay-as-you-go pricing. Instead of buying and maintaining physical servers in a data center, an organisation rents exactly the capacity it needs from a provider such as Google Cloud Platform (GCP), AWS, or Azure, and scales that capacity up or down in minutes. Google Cloud is built on top of the same global infrastructure that powers Google Search, YouTube, and Gmail — one of the largest private networks on Earth.

💡 Layman Explanation

Think of cloud computing like an electricity grid instead of running your own generator. Before the grid existed, every factory needed its own power plant — expensive to build, expensive to maintain, and wasted if under-used. The grid let anyone plug in and pay only for the electricity consumed.

Google Cloud does the same for computing power: Google already built massive data centers connected by its own private fiber-optic backbone around the world. You just "plug in" over the internet, use exactly what you need, and get billed only for that usage — often down to the second.

1.1 Why Cloud Computing?

📝 Theory & Key Points

- Before the cloud, companies practiced **CapEx** (Capital Expenditure) IT — buying racks of physical servers upfront, provisioning for peak load that might only occur a few days a year, and waiting weeks for procurement and installation.
- Cloud computing shifts this to **OpEx** (Operational Expenditure) — you rent resources by the second/minute, so cash isn't locked into idle hardware.
- Elimination of guesswork in capacity planning: if traffic spikes 10x during a sale, a Managed Instance Group's Autoscaler adds VMs automatically; when it settles, VMs are removed.
- Global reach in minutes: a Compute Engine VM can be deployed in Mumbai, Iowa, or Frankfurt within seconds, riding on Google's own private global network rather than the public internet for inter-region traffic.
- Increased speed and agility: new environments (dev/test/prod) that used to take weeks to rack and cable now take minutes via the Cloud Console, gcloud CLI, or Terraform.
- Focus on business value, not "undifferentiated heavy lifting" — patching hypervisors, replacing failed disks, and cooling data centers becomes Google's job, not yours.

1.2 Benefits of Cloud Computing

BENEFIT	WHAT IT MEANS IN PRACTICE
Trade CapEx for OpEx	No upfront hardware purchase; pay per-second (for Compute Engine) like a utility bill.
Economies of scale	Google buys hardware and builds custom infrastructure (e.g. its own network fabric, Jupiter) at a scale no single company could match alone, driving per-unit cost down.
Stop guessing capacity	Scale to actual demand instead of provisioning for a hypothetical peak.
Increase speed & agility	New resources are self-service and available in seconds, encouraging experimentation.
Stop spending on data centers	No racking, stacking, cooling, or physical security to manage.
Go global in minutes	Deploy an application across multiple GCP regions worldwide with a few clicks or one Terraform apply, all connected over Google's private backbone.

1.3 Types of Cloud Computing (Deployment Models)

MODEL	DESCRIPTION	EXAMPLE USE CASE
Public Cloud	Resources owned and operated by a third-party provider (GCP, AWS, Azure) and shared across many tenants via strict isolation.	Startups, SaaS products, most modern workloads.
Private Cloud	Cloud-like resources dedicated to a single organisation, hosted on-premises.	Banks/governments with strict data residency or compliance needs.
Hybrid Cloud	A mix of on-premises infrastructure connected to GCP, typically via Cloud VPN or Cloud Interconnect.	Bursting to the cloud during peak load while keeping sensitive data on-prem.
Multi-Cloud	Workloads deliberately spread across more than one public cloud provider, often unified with Google Anthos/GKE Enterprise.	Avoiding vendor lock-in, or using best-of-breed services from each provider.

1.4 Cloud Service Models

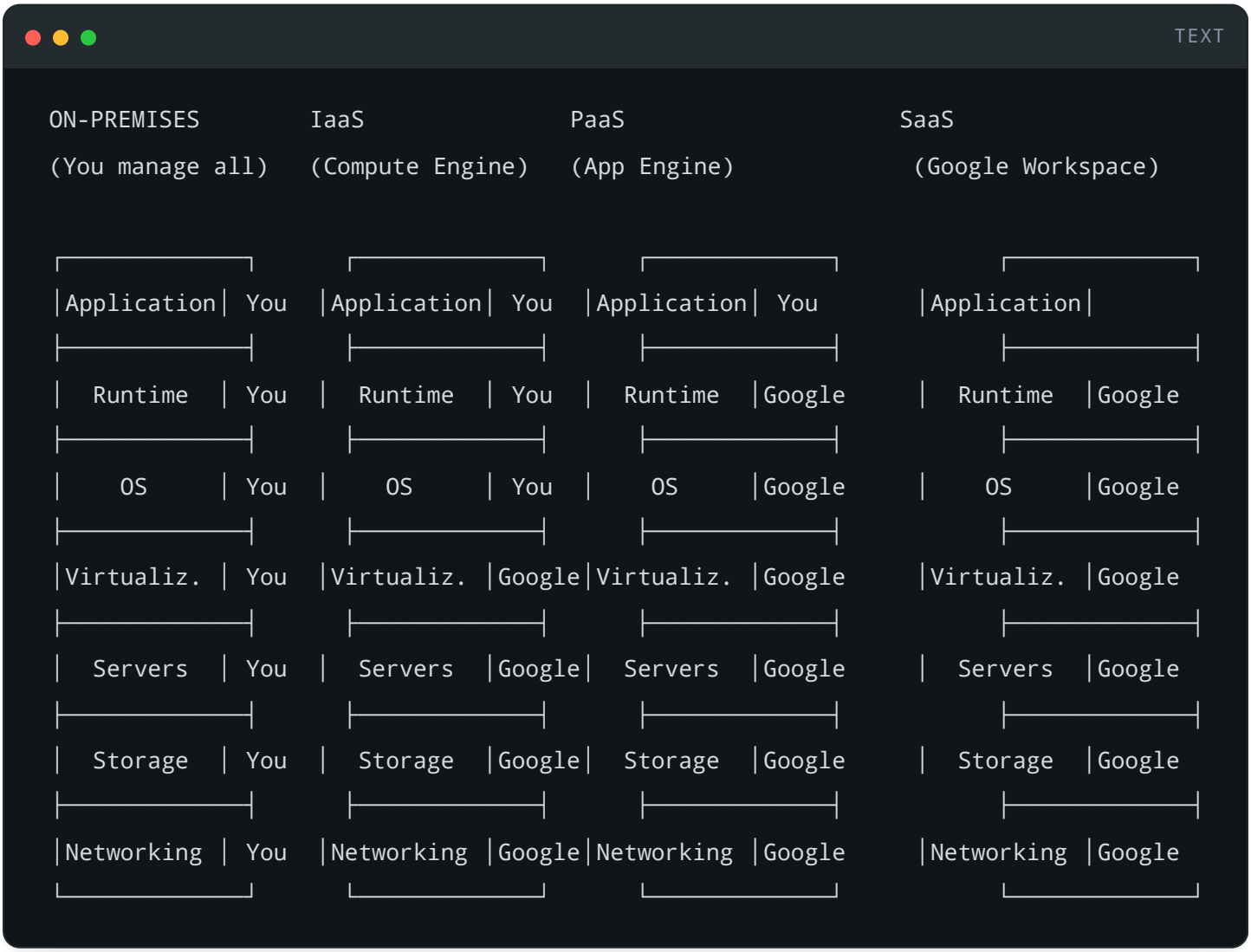


Fig 1: The Cloud Service Model Stack — What You Manage vs What Google Manages

MODEL	YOU MANAGE	GOOGLE MANAGES	GCP EXAMPLE
IaaS (Infrastructure as a Service)	OS, runtime, application, data, patching, firewall rules	Physical servers, hypervisor, storage hardware, network fabric	Compute Engine, Persistent Disk, VPC
PaaS (Platform as a Service)	Application code and data only	OS, runtime, patching, scaling infrastructure, load balancing	App Engine, Cloud Functions, Cloud SQL
SaaS (Software as a Service)	Just your data/configuration inside the app	Everything — the entire application stack	Google Workspace (Gmail, Docs, Drive)

1.5 Scaling in Cloud Computing

📝 Theory & Key Points

- **Vertical Scaling (Scale Up/Down):** increasing or decreasing the size (vCPU/RAM) of a single instance — e.g. changing a VM's machine type from e2-medium to n2-standard-8. Has a ceiling (the largest machine type) and requires a stop/start.
- **Horizontal Scaling (Scale Out/In):** adding or removing more instances behind a Load Balancer. This is the cloud-native pattern and has virtually no ceiling — this is exactly what a GCP Managed Instance Group (MIG) with Autoscaling does automatically.
- Horizontal scaling is generally preferred in the cloud because it improves both capacity AND availability (more instances spread across zones = more fault tolerance), whereas vertical scaling is a single point of failure.
- Autoscaling policies typically trigger on metrics such as CPU utilisation, HTTP load balancing serving capacity, or a custom Cloud Monitoring metric (e.g. queue depth from Pub/Sub).

1.6 Cloud Computing Issues & Challenges

📝 Theory & Key Points

- **Vendor lock-in:** deep use of provider-specific services (e.g. proprietary APIs) makes migrating to another cloud costly. Mitigated with IaC (Terraform) and containerisation (Kubernetes/GKE, which Google itself pioneered as an open standard).
- **Cost management:** the pay-as-you-go model can spiral without governance — orphaned Persistent Disks, oversized VMs, and forgotten test projects are common cost leaks. Google Cloud Billing Reports + Budgets & Alerts + labels address this.
- **Data security & compliance:** data residency laws (GDPR, RBI guidelines, HIPAA) may require data to physically stay within a specific GCP region.
- **Shared Fate Model:** Google's evolution of the traditional Shared Responsibility Model — Google not only secures the underlying cloud but also actively shares in the outcome of the customer's security posture, offering curated security controls, secure-by-default settings, and guidance rather than just drawing a hard line of responsibility.
- **Downtime/outage risk:** even Google Cloud has regional outages — multi-region and multi-zone architectures reduce this risk.

1.7 Security in the Cloud

📝 Theory & Key Points

- Google Cloud's **Shared Fate** approach means identity, access control (IAM), firewall rules, encryption key management, and data classification remain the customer's responsibility, while Google secures the physical data centers, host infrastructure, and — increasingly — provides opinionated secure defaults and blueprints to help customers get security right.
- Defense in depth: identity (Cloud IAM + 2-Step Verification) → network (VPC firewall rules, Cloud Armor) → compute (patching, OS Login) → application (secure coding, Binary Authorization) → data (encryption at rest and in transit, which GCP applies by default on everything).
- Principle of Least Privilege: grant users/service accounts only the exact IAM roles they need — never use the Owner or Editor primitive role for daily operations.
- Security Command Center provides continuous security posture assessment and threat detection across GCP resources — GCP's equivalent of Microsoft Defender for Cloud.

1.8 Cloud Costing Models

PRICING MODEL	DESCRIPTION	BEST FOR
On-Demand (Pay-As-You-Go)	Billed per second (60s minimum) of actual usage, no commitment.	Unpredictable or short-term workloads, dev/test.
Sustained Use Discounts	Automatic discount applied when a VM runs for a significant portion of the billing month — no action needed, GCP applies it for you.	Any workload that happens to run most of the month, no planning required.
Committed Use Discounts (CUDs)	Commit to 1 or 3 years of vCPU/memory usage for a discount of up to ~57%.	Steady-state, predictable production workloads.
Spot VMs	Bid for unused Compute Engine capacity at steep discounts (up to 91%); can be reclaimed by Google with short notice.	Fault-tolerant batch jobs, CI runners, stateless workers.
Free Tier / Always Free	A set of GCP products (e.g. a small e2-micro VM, some Cloud Storage, Cloud Functions invocations) usable at no cost every month, forever.	Learning, prototypes, very small always-on workloads.

🎯 Scenario-Based Interview Questions

Q: Your startup runs a food-delivery app. Traffic is 10x higher on Friday/Saturday nights and near-zero at 4 AM. How would you architect this cost-effectively on GCP, and which scaling approach fits best? Use horizontal scaling via a Managed Instance Group (MIG) with Autoscaling enabled, fronted by a Global External HTTP(S) Load Balancer. Configure an autoscaling policy on CPU utilisation or load-balancing serving capacity so instances scale out automatically before the Friday peak and scale back in overnight. Combine On-Demand pricing for the elastic layer with Committed Use Discounts for the small steady baseline capacity that's always running, and rely on Sustained Use Discounts (applied automatically) plus Cloud Billing budget alerts to catch runaway scale-out. This avoids provisioning for peak 24/7 while still meeting weekend demand.

Q: A finance company asks: 'If Google manages the data center security, why do we still need our own security team on GCP?' Because of Google Cloud's Shared Fate model — Google secures the physical facility, host hypervisor, and underlying network fabric, and actively helps by providing secure-by-default settings and guidance, but the customer is still fully responsible for IAM role assignments, VPC firewall rule configuration, OS-level patching on Compute Engine VMs, data encryption/classification choices, and application-level security. A misconfigured firewall rule or an over-privileged IAM binding (e.g. granting Owner at the project level) is entirely the customer's liability, regardless of how secure Google's data centers are.

GCE

CHAPTER 02 · COMPUTE SERVICES

Compute Engine — Overview & Instance Launch

Compute Engine — images, machine types, and launching your first VM.

Google Compute Engine (GCE) is GCP's core IaaS compute offering — an on-demand, scalable virtual machine service that behaves exactly like a physical server, giving you full control over the OS. Compute Engine is the direct equivalent of AWS EC2 or Azure Virtual Machines, and runs on the same infrastructure and live-migration technology that powers Google's own services.

💡 Layman Explanation

A Compute Engine VM is like renting a fully furnished apartment (a slice of a physical server) instead of building your own house. You choose the size (small studio vs large penthouse = machine type), the neighbourhood (region/zone), and you get the keys (SSH/RDP access) instantly — no construction time. Uniquely, Google can even 'move your furniture to a new apartment while you sleep' — live-migrating your running VM to different physical hardware during maintenance, without you ever noticing a restart.

2.1 Compute Engine — Core Concepts

📝 Theory & Key Points

- A VM instance is defined by: **Image** (the OS template — Debian, Ubuntu, Windows Server, or a custom image), **Machine Type** (vCPU + RAM class), **Boot Disk** (Persistent Disk) + optional additional disks, **Network Interface** (VPC subnet, internal/external IP), and **Zone** (a specific data center within a region).
- GCP's resource hierarchy is **Region → Zone**: a Region (e.g. asia-south1 / Mumbai) contains multiple isolated Zones (e.g. asia-south1-a, -b, -c); a VM lives in exactly one zone.
- A standout Compute Engine feature is **Live Migration** — during host maintenance/hardware failure, Google transparently moves a running VM to different physical hardware with no reboot and typically no perceptible downtime (this is opt-out, not opt-in, unlike most competitors where a maintenance event causes a reboot).
- VMs can be created from Google's public image catalog (Debian, Ubuntu, RHEL, Windows Server, CentOS) or from a custom image/snapshot you built yourself.

CONCEPT	GCP TERM	AWS EQUIVALENT	AZURE EQUIVALENT
Virtual server	Compute Engine Instance (VM)	EC2 Instance	Virtual Machine
OS template	Image	AMI	VM Image
Size class	Machine Type (e.g. e2-medium)	Instance Type (e.g. t3.medium)	VM Size
SSH key auth	Metadata SSH keys / OS Login	Key Pair (.pem)	SSH public key
Firewall	VPC Firewall Rule	Security Group	Network Security Group

2.2 Launching an Instance — Linux VM

BASH

```
# Authenticate and set your default project
gcloud auth login
gcloud config set project my-devops-lab-project

# Create a Debian VM with a public IP
gcloud compute instances create vm-web01 \
  --zone=asia-south1-a \
  --machine-type=e2-medium \
  --image-family=debian-12 \
  --image-project=debian-cloud \
  --tags=http-server,ssh-server

# Create a firewall rule to allow HTTP (port 80) to tagged instances
gcloud compute firewall-rules create allow-http \
  --network=default --direction=INGRESS --action=ALLOW \
  --rules=tcp:80 --target-tags=http-server --source-ranges=0.0.0.0/0

# Get the external IP
gcloud compute instances describe vm-web01 --zone=asia-south1-a \
  --format='get(networkInterfaces[0].accessConfigs[0].natIP)'

# SSH into the VM (gcloud handles key injection + IAM-based auth automatically)
gcloud compute ssh vm-web01 --zone=asia-south1-a
```

2.3 Launching an Instance — Windows VM

BASH

```
# Create a Windows Server VM
gcloud compute instances create vm-win01 \
  --zone=asia-south1-a \
  --machine-type=n2-standard-2 \
  --image-family=windows-2022 \
  --image-project=windows-cloud \
  --tags=rdp-server

# Open RDP port 3389
gcloud compute firewall-rules create allow-rdp \
  --network=default --direction=INGRESS --action=ALLOW \
  --rules=tcp:3389 --target-tags=rdp-server --source-ranges=<your-ip>/32

# Generate a Windows admin password (GCP creates the local user + password for you)
gcloud compute reset-windows-password vm-win01 --zone=asia-south1-a

# Then connect via Remote Desktop using the returned username/password
```

⚠️ Important Warning

- Never leave a firewall rule with source range 0.0.0.0/0 on SSH (22) or RDP (3389) in production — always scope to a known IP range or, better, use Identity-Aware Proxy (IAP) TCP forwarding so no public inbound port needs to be open at all.
- `gcloud compute ssh` relies on OS Login or project/instance metadata SSH keys tied to your IAM identity — prefer OS Login for centrally auditable, IAM-governed SSH access instead of manually managed metadata keys.

🎯 Scenario-Based Interview Questions

Q: Your manager wants a quick dev VM for a 2-hour testing session and wants to minimize cost. Which machine type would you pick, and what should you remember to do afterward?
Pick an E2 machine type such as e2-medium or even e2-small — the E2 family is GCP's cost-optimised general-purpose line, cheaper than N2 for light, bursty workloads, and it's also eligible for Sustained Use Discounts automatically if it happens to run longer than expected. For a strictly time-boxed 2-hour session, an even better option is to launch it as a Spot VM for maximum savings, since a short-lived dev/test session can tolerate the small risk of preemption. Critically, remember to run `gcloud compute instances delete` (not just stop it from the OS) immediately after the session — a stopped VM still incurs Persistent Disk storage charges, and only a fully deleted instance stops billing entirely for compute.

GCEOP

CHAPTER 03 · COMPUTE SERVICES

Compute Engine — Instance Management

Instance lifecycle — OS Login, firewall rules, disks, snapshots, metadata.

Beyond simply launching a VM, real-world operations require managing its lifecycle: choosing the right machine type for the workload, handling SSH key access securely, controlling network traffic with firewall rules, managing disk storage and snapshots, and passing configuration data into the VM at boot time via metadata.

3.1 Instance Types (Machine Families)

FAMILY	OPTIMISED FOR	EXAMPLE MACHINE TYPES
E2	Cost-optimised, general purpose, everyday workloads	e2-micro, e2-medium, e2-standard-4
N2 / N2D	Balanced general purpose (Intel N2 / AMD N2D)	n2-standard-4, n2d-standard-8
C2 / C3	Compute-optimised — high, consistent per-core performance	c2-standard-8
M2 / M3	Memory-optimised — in-memory databases (SAP HANA)	m2-ultramem-208
A2 / G2	GPU-accelerated — ML training/inference, rendering	a2-highgpu-1g, g2-standard-4
T2D	ARM-based (Tau), highly cost-efficient for scale-out workloads	t2d-standard-4

3.2 Key Pair Management

📝 Theory & Key Points

- Unlike AWS/Azure's model of a single named key pair chosen at launch, GCP's default mechanism injects an SSH public key into **instance or project metadata** — the Compute Engine guest agent inside the VM reads this metadata and provisions the corresponding OS user automatically, even after the VM is already running.
- **OS Login** is the recommended enterprise alternative: it ties SSH access directly to a user's Google/IAM identity (via the `roles/compute.osLogin` or `osAdminLogin` role) instead of a shared metadata key, giving centralised, auditable, and instantly revocable access — disable a user's IAM access and their SSH access disappears immediately, with no manual key cleanup on every VM.
- `gcloud compute ssh` transparently handles ephemeral key generation and injection for you in the metadata-key model, which is why it 'just works' without manual `ssh-keygen` steps in typical use.

BASH

```
# Enable OS Login at the project level (recommended for teams)
gcloud compute project-info add-metadata --metadata enable-oslogin=TRUE

# Grant a user permission to SSH via OS Login
gcloud projects add-iam-policy-binding my-devops-lab-project \
  --member="user:veera@example.com" --role="roles/compute.osLogin"

# Manually add an SSH key to project-wide metadata (legacy approach)
gcloud compute project-info add-metadata \
  --metadata-from-file ssh-keys=~/.ssh/my_key.pub
```

3.3 Firewall Rules (Security Groups Equivalent)

📝 Theory & Key Points

- GCP **Firewall Rules** are stateful, and — critically different from Azure/AWS — they attach to the **VPC network itself** (a global resource), not to a specific subnet or NIC. Rules apply to instances based on **Network Tags** or **Service Accounts**, letting you target 'all web servers' regardless of which subnet they're in.
- Every VPC has two implied rules by default: **deny all ingress** and **allow all egress** — nothing gets in until you explicitly allow it.
- Rules are evaluated by **Priority** (0-65535, lower number = higher priority evaluated first); Deny rules and Allow rules can be mixed and the most specific/highest-priority match wins.
- Best practice: target rules using Service Accounts rather than Network Tags where possible — tags can be set by anyone with edit permission on the instance, while Service Account attachment is a more tightly IAM-controlled boundary.

BASH

```
# Allow inbound HTTPS only from a specific IP range, targeting a service account
gcloud compute firewall-rules create allow-https-web \
  --network=default --direction=INGRESS --action=ALLOW \
  --rules=tcp:443 \
  --source-ranges=203.0.113.0/24 \
  --target-service-accounts=web-tier@my-devops-lab-project.iam.gserviceaccount.com

# Allow SSH only via Identity-Aware Proxy's IP range (no public SSH exposure at all)
gcloud compute firewall-rules create allow-iap-ssh \
  --network=default --direction=INGRESS --action=ALLOW \
  --rules=tcp:22 --source-ranges=35.235.240.0/20

# List all firewall rules in the project
gcloud compute firewall-rules list
```

3.4 Storage and Snapshots

📝 Theory & Key Points

- Every VM has a **Boot Disk** (a Persistent Disk) and can attach additional **Persistent Disks** for application data, independent of the VM's lifecycle.
- A **Snapshot** is an incremental, point-in-time backup of a Persistent Disk stored in Cloud Storage — only changed blocks since the last snapshot are billed, making frequent snapshots cost-efficient.
- Disk performance tiers: Standard PD (HDD-backed, cheapest), Balanced PD (SSD-backed, good default), SSD PD (highest consistent IOPS), and Extreme PD (highest performance, tunable IOPS for demanding databases).

BASH

```
# Attach a new 128GB balanced Persistent Disk to a running VM
gcloud compute disks create disk-appdata --zone=asia-south1-a \
  --size=128GB --type=pd-balanced
gcloud compute instances attach-disk vm-app01 --zone=asia-south1-a \
  --disk=disk-appdata

# Take a snapshot of a disk for backup
gcloud compute disks snapshot disk-appdata --zone=asia-south1-a \
  --snapshot-names=snap-appdata-backup

# Create a new disk from that snapshot (e.g. to clone into another VM)
gcloud compute disks create disk-restored --zone=asia-south1-a \
  --source-snapshot=snap-appdata-backup
```

3.5 User Data and Metadata

📝 Theory & Key Points

- **Startup Scripts** are the GCP equivalent of AWS User Data / Azure Custom Data — a script (Bash for Linux, PowerShell for Windows) passed as instance metadata that runs automatically on every boot, letting you bootstrap software with zero manual login.
- The **Metadata Server** is a REST endpoint reachable only from inside the VM at `http://metadata.google.internal/computeMetadata/v1/` that lets an application query its own project ID, zone, tags, and custom metadata at runtime.
- The metadata server is also how a VM retrieves a short-lived OAuth 2.0 access token for its attached Service Account, letting applications call other GCP APIs without any embedded credentials.

BASH

```
# Create a VM with a startup script that auto-installs and starts NGINX
cat > startup.sh <<'EOF'
#!/bin/bash
apt-get update
apt-get install -y nginx
systemctl enable nginx
systemctl start nginx
EOF

gcloud compute instances create vm-web02 --zone=asia-south1-a \
  --machine-type=e2-medium --image-family=debian-12 --image-project=debian-cloud \
  --metadata-from-file startup-script=startup.sh

# Query instance metadata from INSIDE the VM (note the required header)
curl "http://metadata.google.internal/computeMetadata/v1/instance/zone" \
  -H "Metadata-Flavor: Google"
```

🎯 Scenario-Based Interview Questions

Q: Your web tier VMs need outbound-only internet access for OS updates, but must never accept unsolicited inbound connections from the internet — only from an internal Load Balancer's health-check ranges. How do you configure this on GCP? Create a firewall rule targeting the web tier's Service Account (or network tag) with an INGRESS Allow rule scoped to Google's known health-check source ranges (130.211.0.0/22 and 35.191.0.0/16) plus the internal Load Balancer's proxy subnet, on the application port. Leave the default implied 'allow all egress' rule in place so outbound OS-update traffic still works. Because GCP firewall rules are stateful, response traffic to those outbound requests is automatically permitted without needing an explicit inbound rule — inbound stays scoped to only the health-checker and load balancer ranges, never the open internet.

LB

CHAPTER 04 · COMPUTE SERVICES

Load Balancing

Distributing traffic — global and regional HTTP(S) and network load balancers.

A Load Balancer distributes incoming network traffic across multiple backend VM instances so that no single instance is overwhelmed, and so that if one instance fails, traffic is automatically routed to the healthy ones. GCP's load balancing portfolio is unusually powerful because it's built on Google's own global network — a single anycast IP address can front backends in multiple regions worldwide.

💡 Layman Explanation

A load balancer is like the host at a busy restaurant who seats new customers at whichever table is free, instead of letting everyone crowd around one table while others sit empty. GCP's global load balancers take this further: imagine one restaurant host who can instantly redirect customers to the nearest branch of the restaurant chain, anywhere in the world, using private roads (Google's network) instead of public traffic.

4.1 Types of GCP Load Balancers

LOAD BALANCER TYPE	SCOPE	OSI LAYER	USE CASE
Global External HTTP(S) LB	Global (anycast IP)	Layer 7	Internet-facing web apps/APIs needing a single global IP, URL-path routing, and CDN integration.
Regional External HTTP(S) LB	Regional	Layer 7	HTTP(S) traffic that must stay within one region for compliance reasons.
Internal HTTP(S) LB	Regional, internal	Layer 7	Microservice-to-microservice traffic inside a VPC, never touching the internet.
External/Internal TCP/UDP Network LB	Regional	Layer 4	Non-HTTP protocols, extreme throughput, pass-through load balancing that preserves the client's source IP.

4.2 Core Load Balancing Concepts

📝 Theory & Key Points

- A **Backend Service** defines which Managed Instance Group(s) or Network Endpoint Groups (NEGs) receive traffic, along with the balancing mode (e.g. utilisation, rate).
- A **Health Check** continuously probes each backend instance (e.g. HTTP GET /healthz every few seconds); instances that fail are automatically removed from rotation until healthy again.
- A **URL Map** (for HTTP(S) load balancers) lets you route different URL paths to different backend services — e.g. /api/* to one backend, /images/* to a Cloud Storage bucket acting as a backend.
- **Global HTTP(S) Load Balancing** uses a single **anycast IP** announced from many Google edge locations simultaneously — a user's request automatically enters Google's network at the nearest edge point of presence and travels the rest of the way over Google's private backbone, not the public internet.

BASH

```
# Create a health check
gcloud compute health-checks create http web-health-check --port=80 --request-
path=/healthz

# Create a backend service and attach a Managed Instance Group to it
gcloud compute backend-services create web-backend-service \
  --protocol=HTTP --health-checks=web-health-check --global

gcloud compute backend-services add-backend web-backend-service \
  --instance-group=web-mig --instance-group-zone=asia-south1-a --global

# Create a URL map, HTTP proxy, and global forwarding rule (the frontend IP)
gcloud compute url-maps create web-url-map --default-service=web-backend-service
gcloud compute target-http-proxies create web-http-proxy --url-map=web-url-map
gcloud compute forwarding-rules create web-http-rule --global \
  --target-http-proxy=web-http-proxy --ports=80
```

🎯 Scenario-Based Interview Questions

Q: Your 3-tier app has a public web tier and an internal API tier that the database team says must never be internet-reachable. How do you load-balance traffic to the API tier while enforcing that isolation on GCP? Deploy an Internal HTTP(S) Load Balancer for the API tier — its forwarding rule only gets an internal IP address inside the VPC, so it is structurally unreachable from the internet regardless of firewall configuration. The web tier calls the internal load balancer's private IP or internal DNS name for API requests. Combine this with a firewall rule on the API tier's Service Account that only allows inbound traffic from the web tier's Service Account (or the internal LB's health-check ranges plus the web tier's subnet), giving defense in depth rather than relying on a single control.

MON

CHAPTER 05 · COMPUTE SERVICES

Billing and Monitoring

Cloud Billing and Cloud Monitoring — budgets, metrics, alerts, logging.

Cloud cost visibility and operational monitoring are two sides of the same coin: monitoring tells you if your systems are healthy, while cost management tells you if you're paying a sensible amount to keep them that way. GCP provides Cloud Monitoring and Cloud Logging (formerly bundled as 'Stackdriver') for observability, and Cloud Billing for financial governance.

5.1 Cloud Billing

📝 Theory & Key Points

- Every GCP **Project** is linked to a **Billing Account**, which can be shared across many projects — costs roll up and can be broken down per-project, per-service, or per-**Label** (GCP's equivalent of Azure/AWS tags).
- **Budgets & Alerts** can be set per billing account or project, with automated email/Pub/Sub notifications when spend crosses a percentage threshold (e.g. 50%, 90%, 100% of a monthly budget) — the Pub/Sub integration even allows fully automated responses, like disabling billing on a runaway project.
- **Labels** (key-value pairs like `env=prod`, `team=devops`) attached to resources are essential for cost allocation/chargeback across teams — unlabeled resources are the #1 cause of 'mystery spend'.
- The **Recommender** service (Active Assist) proactively flags cost-saving opportunities — e.g. an idle VM that could be stopped, or a Persistent Disk that's unattached but still being billed.
- Billing export to **BigQuery** is the standard way to do detailed, ad-hoc cost analysis with SQL across very large organisations.

5.2 Cloud Monitoring & Alerting

📝 Theory & Key Points

- **Cloud Monitoring** is the umbrella observability platform collecting metrics (numeric time-series like CPU utilisation) across every GCP resource, automatically, with no setup required for most infrastructure metrics.
- **Cloud Logging** ingests logs from every GCP service, Compute Engine VMs (via the Ops Agent), and application code — queryable with a powerful structured query language directly in the Logs Explorer.
- **Alerting Policies** are rules defined on a metric or log-based metric that trigger a **Notification Channel** (email, SMS, Slack, PagerDuty, webhook, Pub/Sub) when a threshold is breached.
- **Cloud Trace** and **Cloud Profiler** round out GCP's Application Performance Monitoring (APM) story — distributed tracing across microservices and continuous CPU/memory profiling in production.
- **Uptime Checks** proactively probe a public endpoint from multiple global locations to catch outages before users report them.

BASH

```
# Create a budget with an alert at 90% of a $500/month budget
gcloud billing budgets create \
  --billing-account=XXXXXX-XXXXXX-XXXXXX \
  --display-name="Monthly DevOps Budget" \
  --budget-amount=500 \
  --threshold-rule=percent=0.9

# Create a metric-based alerting policy: fire if average CPU > 85% for 5 minutes
gcloud alpha monitoring policies create \
  --notification-channels=CHANNEL_ID \
  --display-name="High CPU Alert" \
  --condition-display-name="CPU > 85%" \
  --condition-filter='resource.type="gce_instance" AND
metric.type="compute.googleapis.com/instance/cpu/utilization"' \
  --condition-threshold-value=0.85 --condition-threshold-duration=300s

# Cloud Logging query: find the top errors in the last 24 hours
gcloud logging read \
  'severity>=ERROR AND timestamp>="2026-08-15T00:00:00Z"' \
  --limit=20 --format=json
```

🎯 Scenario-Based Interview Questions

Q: Finance reports the GCP bill jumped 40% this month with no corresponding traffic increase. How do you investigate? Start in Cloud Billing → Reports, grouped by Project and then by SKU/Service, comparing this month vs last. Look for anomalies: a common cause is orphaned resources — unattached Persistent Disks or reserved external IPs left after a VM was deleted, forgotten dev/test VMs left running 24/7, or a Managed Instance Group autoscaler that scaled out and never scaled back in due to a misconfigured cooldown period. Cross-check the Recommender/Active Assist panel for idle-resource recommendations, and review Cloud Audit Logs (Admin Activity logs) for who created new resources and when. Once identified, delete the waste and enforce a labeling + budget-alert policy so it's caught within days next time, not a full billing cycle.

SCALE

CHAPTER 06 · COMPUTE SERVICES

Autoscaler

Managed Instance Groups and reactive/scheduled/queue-based autoscaling.

Autoscaling automatically adjusts the number of running compute instances in response to real-time demand, ensuring an application has enough capacity to handle load spikes while avoiding the cost of running excess capacity during quiet periods. On GCP, this is implemented through Managed Instance Groups (MIGs).

6.1 Managed Instance Groups (MIGs)

📝 Theory & Key Points

- A **Managed Instance Group** is a group of identical, load-balanced VM instances created from a single **Instance Template** (machine type + image + startup script + metadata), managed as one logical unit — GCP's equivalent of an AWS Auto Scaling Group or Azure VM Scale Set.
- A **Regional MIG** automatically spreads instances across multiple zones within a region for higher resilience against a single zone-level failure — the recommended default for production.
- MIGs provide **self-healing**: if a health check determines an instance is unhealthy, the MIG automatically recreates it — this works even without autoscaling enabled, purely as a reliability feature.

6.2 Scaling Types and Policies

SCALING TYPE	TRIGGER	EXAMPLE
Manual Scaling	Human sets a fixed instance count	Set target size to 5 before a planned marketing campaign.
Metric-Based (Reactive) Autoscaling	A metric crosses a threshold (CPU utilisation, load balancing serving capacity, Cloud Monitoring custom metric)	Scale out when average CPU > 75% for the MIG.
Schedule-Based Autoscaling	A defined schedule with a minimum required instance count for specific time windows	Guarantee at least 10 instances every weekday 9 AM–8 PM.
Queue-Based Autoscaling	A custom Cloud Monitoring metric fed by, e.g., Pub/Sub queue depth	Scale worker VMs based on the number of unprocessed messages in a Pub/Sub subscription.

6.3 GCP Auto-Scaling — Configuration

BASH

```
# Create an instance template
gcloud compute instance-templates create web-template \
  --machine-type=e2-medium --image-family=debian-12 --image-project=debian-cloud \
  --metadata-from-file startup-script=startup.sh --tags=http-server

# Create a regional Managed Instance Group from the template
gcloud compute instance-groups managed create web-mig \
  --template=web-template --size=2 --region=asia-south1

# Attach an autoscaling policy: 2 to 10 instances, target 75% CPU utilisation
gcloud compute instance-groups managed set-autoscaling web-mig \
  --region=asia-south1 --min-num-replicas=2 --max-num-replicas=10 \
  --target-cpu-utilization=0.75 --cool-down-period=90

# Watch the MIG scale in real time
gcloud compute instance-groups managed describe web-mig --region=asia-south1
```

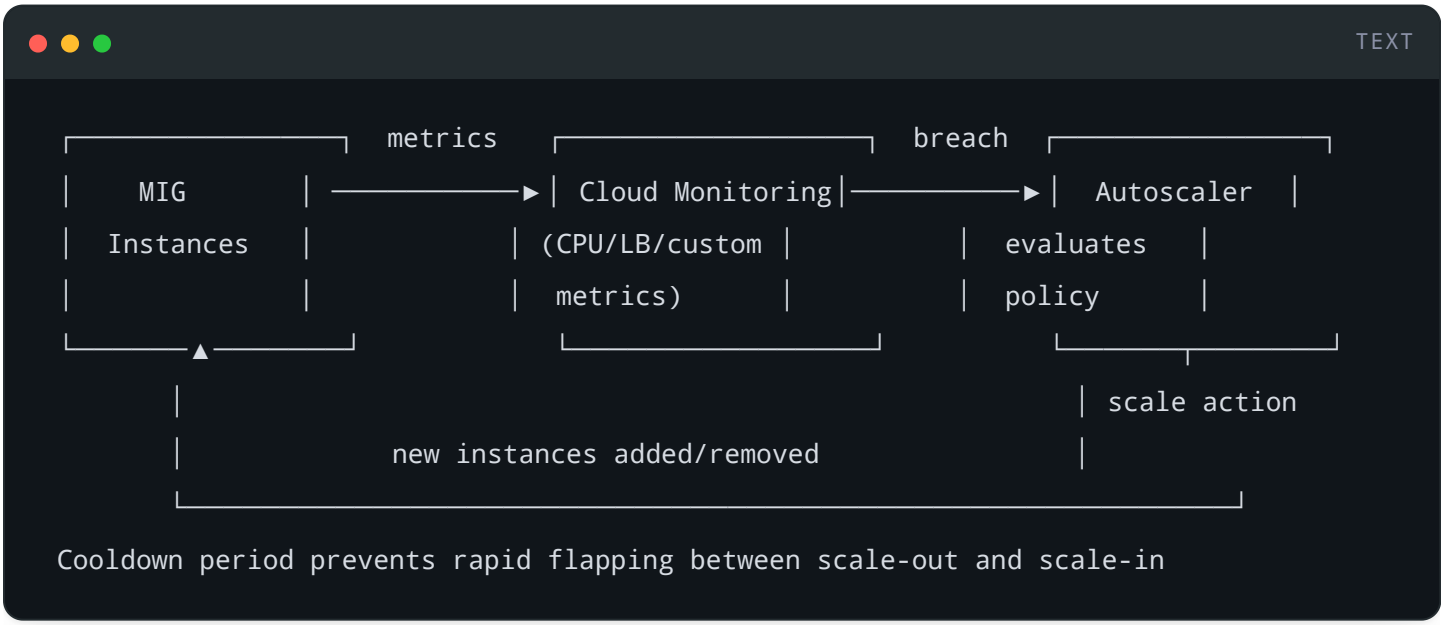


Fig 2: Reactive Autoscale Feedback Loop

⚠️ Important Warning

- Always set a sensible cool-down period between scale actions — without it, a fluctuating metric can cause 'flapping' (rapid scale out/in), which is both costly and destabilising.
- Scaling in (removing instances) respects connection draining timeout settings so in-flight requests aren't dropped mid-response.

🎯 Scenario-Based Interview Questions

Q: An e-commerce site knows it gets a predictable traffic spike every day at 6 PM when people shop after work, but reactive CPU-based autoscaling always lags a few minutes behind the spike, causing brief slowness. How do you fix this on GCP? Combine schedule-based autoscaling with the existing reactive CPU policy instead of relying on CPU alone. Add a scheduled scaling configuration that sets a higher minimum required instances count (e.g. from 2 to 6) starting at 5:45 PM, ahead of the known 6 PM spike, so capacity is already in place before demand arrives rather than reacting after CPU crosses the threshold. Keep the reactive CPU-utilisation policy active as a safety net for unpredictable spikes on top of the scheduled baseline, and let the schedule relax back to the lower minimum late at night.

DISK

CHAPTER 07 · STORAGE SERVICES

Storage Services — Persistent Disk & Cloud Filestore

Persistent Disk and Cloud Filestore — block storage vs shared NFS.

Compute needs persistent storage attached to it. GCP offers two distinct storage services for this: Persistent Disk (block storage attached to a single VM, equivalent to AWS EBS) and Cloud Filestore (managed NFS file shares accessible by many VMs simultaneously, equivalent to AWS EFS).

7.1 Persistent Disk (EBS Equivalent)

📝 Theory & Key Points

- A **Persistent Disk (PD)** is durable block storage, network-attached (not physically inside the VM's host), that survives independently of any single VM's lifecycle — deleting a VM does not delete its disks unless explicitly configured to do so.
- PDs can be resized live without downtime, and — uniquely among major clouds — a PD can be attached in **read-only mode to multiple VMs simultaneously** for use cases like serving a shared, static dataset.
- **Regional Persistent Disks** synchronously replicate data across two zones in the same region, providing protection against a single zone failure without needing application-level replication.

DISK TYPE	PERFORMANCE	TYPICAL USE
Standard PD (pd-standard)	HDD-backed, lowest cost, higher latency	Backups, infrequently accessed data, large sequential I/O
Balanced PD (pd-balanced)	SSD-backed, good price/performance balance	Default choice for most production boot disks and general workloads
SSD PD (pd-ssd)	SSD-backed, high consistent IOPS/throughput	Databases, latency-sensitive applications
Extreme PD (pd-extreme)	Highest performance, independently provisioned IOPS	SAP HANA, large transactional databases with strict IOPS requirements

7.2 Cloud Filestore (EFS Equivalent)

📝 Theory & Key Points

- **Cloud Filestore** provides fully managed NFS file shares accessible concurrently by many VMs (or GKE pods) at once — ideal for shared configuration, shared web content across a scaled-out web tier, or lift-and-shift of legacy apps expecting a network file server.
- Unlike a Persistent Disk, Filestore is shared network storage using the NFSv3 protocol, mountable from multiple Compute Engine VMs or GKE clusters simultaneously.
- Service tiers: Basic HDD (cost-effective), Basic SSD (higher performance), Zonal/Enterprise (higher availability with replication) — chosen based on the throughput and availability the workload needs.

ASPECT	PERSISTENT DISK (BLOCK STORAGE)	CLOUD FILESTORE (NETWORK FILE SHARE)
AWS Equivalent	EBS	EFS
Attach model	One VM at a time (or read-only to many)	Many clients concurrently, read-write
Protocol	Block device (mounted as a raw disk)	NFSv3
Typical use	Boot disk, database data files	Shared config, shared uploads, lift-and-shift file servers

BASH

```
# Create a 256GB SSD Persistent Disk and attach it to a VM
gcloud compute disks create disk-db-data --zone=asia-south1-a \
  --size=256GB --type=pd-ssd
gcloud compute instances attach-disk vm-db01 --zone=asia-south1-a \
  --disk=disk-db-data

# Create a Filestore instance (1TB Basic HDD tier)
gcloud filestore instances create nfs-shared-config \
  --zone=asia-south1-a --tier=BASIC_HDD \
  --file-share=name="shareddata",capacity=1TB \
  --network=name="default"

# Mount the Filestore share on a Linux VM
sudo apt-get install -y nfs-common
sudo mkdir -p /mnt/config
sudo mount <filestore-ip>:/shareddata /mnt/config
```

🎯 Scenario-Based Interview Questions

Q: You're migrating an on-prem 3-node web farm that all read/write to a shared 'uploads' folder on a Linux NFS server. What's the right GCP storage choice, and why not a Persistent Disk?
Cloud Filestore is the correct choice, mounted via NFS and attached to all three Compute Engine VMs simultaneously, exactly replicating the shared-folder behaviour they rely on today. A standard read-write Persistent Disk cannot serve this use case because it attaches to only one VM at a time in read-write mode — using it would force all uploads through a single VM, breaking the multi-node architecture. Filestore's managed NFS handles concurrent multi-writer access natively, with no cluster filesystem software to install or maintain.

VPC

CHAPTER 08 · NETWORKING

Networking — VPC Network

VPC Network fundamentals — a global resource, regional subnets, CIDR.

A Virtual Private Cloud (VPC) network is the fundamental building block of your private network in GCP — a logically isolated, software-defined network where you launch VMs and other resources. GCP's VPC has one crucial architectural difference from AWS/Azure: it is a GLOBAL resource, not regional.

💡 Layman Explanation

In AWS or Azure, a VPC/VNET is like one private office building in one city — if you want offices in another city, you build a separate building and connect them with a bridge (peering). In GCP, a VPC is more like a single company that already has private offices in every city worldwide, all connected by the company's own private roads (Google's global network) — you just open a new branch office (a subnet) in whichever city (region) you need, and it's automatically part of the same private network, no bridge required.

8.1 Networking Basics

📝 Theory & Key Points

- A GCP **VPC Network** is a **global** resource — it spans all GCP regions automatically. **Subnets**, however, are **regional** resources, each defined by a CIDR range within a single region.
- This means two VMs in different regions but the same VPC (in different regional subnets) can communicate over their private IPs directly across Google's backbone, with no VPC peering required — a major structural difference from AWS/Azure where cross-region communication within the 'same' network requires peering or a transit gateway.
- GCP supports two VPC modes: **Auto Mode** (a subnet is automatically created in every region with predefined ranges — convenient for quick starts) and **Custom Mode** (you define every subnet's region and CIDR range explicitly — the recommended choice for production).
- VPC networks are private by default — no traffic flows in or out until you explicitly configure firewall rules, external IPs, or Cloud NAT.

8.2 Public IPs vs Private IPs

TYPE	REACHABLE FROM	TYPICAL USE
Private (Internal) IP	Only within the VPC (or connected networks via VPN/Interconnect/peering)	Internal communication between app tiers, databases
Public (External) IP	The internet	Load balancer frontends, bastion hosts, public-facing endpoints

8.3 CIDR Range Basics

CIDR	SUBNET MASK	TOTAL ADDRESSES	TYPICAL USE
/16	255.255.0.0	65,536	A large regional subnet with room for major future growth
/24	255.255.255.0	256	A standard-sized application-tier subnet
/28	255.255.255.240	16	A small subnet, e.g. for a NAT gateway or a handful of management VMs

8.4 Subnet Calculation Example

```text
VPC NETWORK: vpc-prod   (a single GLOBAL resource — no per-region VPC needed)

┌──────────────────────────────────────────────────────────────────────┐
│ Subnet: snet-web-asia   10.0.1.0/24  (asia-south1)                   │
├──────────────────────────────────────────────────────────────────────┤
│ Subnet: snet-web-us     10.1.1.0/24  (us-central1)                   │
├──────────────────────────────────────────────────────────────────────┤
│ Subnet: snet-web-eu     10.2.1.0/24  (europe-west1)                  │
└──────────────────────────────────────────────────────────────────────┘

All three subnets are part of ONE VPC — VMs across regions talk
over their PRIVATE IPs directly, no peering needed, using Google's
own global backbone network for the cross-region hop.
```

Fig 3: One Global VPC, Regional Subnets Spanning the World

8.5 Creating Custom VPCs and Subnets

```bash
# Create a custom-mode VPC (no auto-created subnets)
gcloud compute networks create vpc-prod --subnet-mode=custom

# Add regional subnets in different parts of the world, same VPC
gcloud compute networks subnets create snet-web-asia \
  --network=vpc-prod --region=asia-south1 --range=10.0.1.0/24

gcloud compute networks subnets create snet-web-us \
  --network=vpc-prod --region=us-central1 --range=10.1.1.0/24

# List all subnets across the VPC
gcloud compute networks subnets list --network=vpc-prod
```

🎯 Scenario-Based Interview Questions

Q: You are designing a VPC for a global application with users in India, the US, and Europe, needing app servers in all three regions that can talk to a central database over private IPs. How does GCP's VPC model simplify this compared to AWS/Azure? Because a GCP VPC is a single global resource, you create ONE VPC (e.g. vpc-prod) with three regional subnets — one in asia-south1, one in us-central1, one in europe-west1. App servers in any of the three regions and the central database (wherever it's placed) can all communicate over private internal IPs immediately, riding Google's private backbone for the cross-region hops, with zero VPC peering configuration required. In AWS or Azure, achieving the same result would require three separate regional VPCs/VNETs plus explicit peering connections (or a transit gateway/hub) between every pair of regions — GCP's global-VPC model removes that entire layer of complexity by design.

VPCC

CHAPTER 09 · NETWORKING

Networking — VPC Controls

Firewall rules, routes, Cloud NAT, VPN/Interconnect, hub-and-spoke.

Once a VPC and its subnets exist, two mechanisms control how traffic actually flows: Firewall Rules (filtering, covered in depth in Module 3) and Routes + Cloud Router (directing traffic to its destination, including out to the internet or back to on-premises).

9.1 Firewall Rules — ACL Equivalent (Recap)

📝 Theory & Key Points

- As covered in Module 3, GCP Firewall Rules function like a stateful Access Control List / Security Group — but attached to the VPC (global) rather than a subnet or NIC, and targeted using Network Tags or Service Accounts rather than being bound to a specific resource.
- **Hierarchical Firewall Policies** can additionally be attached at the Organization or Folder level, letting a central security team enforce non-overridable baseline rules (e.g. 'never allow inbound SSH from 0.0.0.0/0, anywhere in the company') across every project and VPC beneath them.

9.2 Routes & Cloud Router

📝 Theory & Key Points

- GCP automatically creates **System-Generated Routes** so traffic between subnets in the same VPC, and outbound to the internet (if a VM has an external IP), works without configuration.
- A **Custom Static Route** lets you override this default behaviour — e.g. force all outbound traffic from a subnet through a central Network Virtual Appliance (firewall) instead of going directly to the internet, using a 'next hop' pointing at that instance.
- **Cloud VPN** connects your VPC to an on-premises network over an encrypted IPsec tunnel across the public internet.
- **Cloud Interconnect** (Dedicated or Partner) connects on-prem to Google's network over a private, dedicated circuit (not traversing the public internet) — used when consistent low latency and higher bandwidth are required (GCP's equivalent of AWS Direct Connect / Azure ExpressRoute).
- **Cloud Router** is the control-plane component that dynamically exchanges routes using BGP over a VPN or Interconnect connection, and is also required to provision a **Cloud NAT** gateway.
- **Cloud NAT** gives an entire subnet outbound-only internet access (for OS updates, package downloads) without giving individual VMs their own external IP — a managed, distributed NAT service with no single point of failure or bandwidth bottleneck.

TEXT

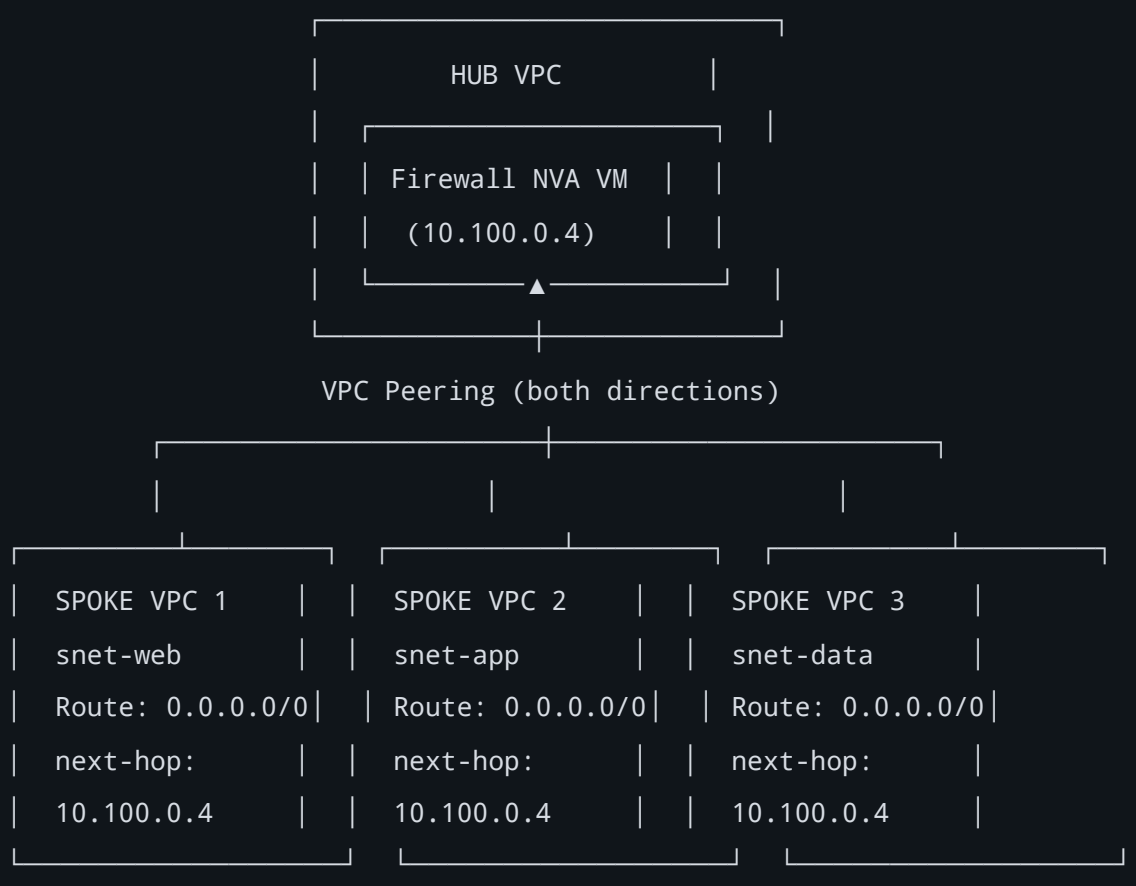


Fig 4: Hub-and-Spoke Network with a Custom Route Forcing Traffic Through a Firewall Appliance

BASH

```
# Create a Cloud Router (required for Cloud NAT)
gcloud compute routers create nat-router --network=vpc-prod --region=asia-south1

# Create a Cloud NAT gateway so private VMs get outbound internet, no public IP needed
gcloud compute routers nats create nat-config \
  --router=nat-router --region=asia-south1 \
  --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges

# Create a custom static route forcing traffic through a central firewall appliance
gcloud compute routes create route-via-firewall \
  --network=vpc-prod --destination-range=0.0.0.0/0 \
  --next-hop-instance=nva-firewall --next-hop-instance-zone=asia-south1-a \
  --priority=100
```

🎯 Scenario-Based Interview Questions

Q: Your private Compute Engine VMs (no external IPs, by security policy) need to download OS security patches from the internet. How do you give them outbound internet access without assigning any public IPs? Provision a Cloud Router in the VPC's region, then create a Cloud NAT gateway attached to that router, covering the relevant subnets. Cloud NAT gives every VM in those subnets outbound-only internet connectivity (translating private IPs to a pool of external IPs Google manages) with zero public IP assigned to any VM itself — inbound connections from the internet remain completely impossible, satisfying the no-public-IP security policy while still letting apt-get/yum update commands reach the internet.

GCS

CHAPTER 10 · OBJECT STORAGE

Object Storage — Cloud Storage

Cloud Storage fundamentals — buckets, storage classes, versioning, signed URLs.

Google Cloud Storage (GCS) is GCP's massively scalable object storage service for unstructured data — images, videos, log files, backups, and static website assets. It is the direct equivalent of AWS S3 or Azure Blob Storage, storing objects inside flat containers called Buckets.

💡 Layman Explanation

Think of Cloud Storage as an infinitely large warehouse of labelled boxes. Each box (object) has a unique label (its name/path) inside a section (bucket), and you can retrieve any box instantly by its label — regardless of whether the warehouse holds 10 boxes or 10 billion.

10.1 What is Cloud Storage?

📝 Theory & Key Points

- A **Bucket** is the top-level container for objects — and unlike Azure (where storage account names are the globally unique boundary), GCS **bucket names are globally unique across all of GCP**, exactly like AWS S3 bucket names.
- Objects are stored in a flat namespace (no true folders), though object names can include forward slashes (e.g. images/2026/aug/photo.jpg) to simulate a folder hierarchy in the Console and gsutil/gcloud storage tools.
- Every bucket has a **Location Type** chosen at creation: **Region** (data in one specific region, lowest latency for co-located compute), **Dual-region** (data replicated between two specific regions), or **Multi-region** (data replicated across a large geographic area like 'asia' or 'us' for maximum availability).
- Every object is automatically versioned by generation number internally, and can additionally have full **Object Versioning** enabled to keep historical versions accessible after being overwritten or deleted.

GCP TERM	AWS S3 EQUIVALENT	AZURE BLOB EQUIVALENT
Bucket (globally unique name)	Bucket	Container (namespaced under a Storage Account)
Object	Object	Blob
Storage Class	Storage Class	Access Tier
Signed URL	Pre-signed URL	Shared Access Signature (SAS)
Object Lifecycle Management	S3 Lifecycle Rule	Lifecycle Management Policy

10.2 Storage Classes

STORAGE CLASS	ACCESS PATTERN	COST PROFILE	MIN. STORAGE DURATION
Standard	Frequently accessed, 'hot' data	Highest storage cost, no retrieval fee	None
Nearline	Accessed roughly once a month	Lower storage cost, small retrieval fee	30 days
Coldline	Accessed roughly once a quarter	Lower still, higher retrieval fee	90 days
Archive	Accessed less than once a year, long-term retention/compliance	Lowest storage cost, highest retrieval fee, retrieval still near-instant (no rehydration delay unlike Azure Archive)	365 days

10.3 Versioning

📝 Theory & Key Points

- When **Object Versioning** is enabled on a bucket, every overwrite or delete keeps the prior version (a 'noncurrent version') accessible by its generation number rather than destroying it.
- Combine with an **Object Lifecycle Rule** to automatically clean up old noncurrent versions after a set number of days or once a set number of newer versions exist, avoiding unbounded storage growth from keeping every version forever.
- **Soft Delete** (a newer GCS feature) additionally retains recently deleted objects for a configurable retention window by default, protecting against accidental bulk deletion even without full versioning enabled.

10.4 Bucket ACL and Permissions

📝 Theory & Key Points

- GCS supports two permission systems: **Uniform Bucket-Level Access** (recommended — access controlled purely through Cloud IAM at the bucket level, simple and consistent) and legacy **Fine-Grained ACLs** (per-object Access Control Lists, more granular but harder to audit).
- By default, buckets are **private** — no anonymous access, every request must be authenticated (a user/service account's IAM permissions, or a Signed URL).
- A bucket can optionally be made public by granting the `allUsers` special identity the Storage Object Viewer role — commonly used for static website assets, never for anything containing sensitive data.
- **Signed URLs** grant time-limited, scope-limited access (e.g. read-only, expires in 1 hour) without granting the requester any IAM role at all — the standard way to give external users temporary access, equivalent to an AWS S3 pre-signed URL or Azure SAS token.

BASH

```bash
# Create a bucket (name must be globally unique across ALL of GCP)
gcloud storage buckets create gs://devopslab-uploads-2026 \
  --location=asia-south1 --uniform-bucket-level-access

# Upload an object
gcloud storage cp ./aug2026.pdf gs://devopslab-uploads-2026/reports/aug2026.pdf

# Enable versioning on the bucket
gcloud storage buckets update gs://devopslab-uploads-2026 --versioning

# Grant a specific user read-only IAM access to the bucket
gcloud storage buckets add-iam-policy-binding gs://devopslab-uploads-2026 \
  --member="user:partner@example.com" --role="roles/storage.objectViewer"

# Generate a time-limited signed URL (read-only, expires in 1 hour)
gcloud storage sign-url gs://devopslab-uploads-2026/reports/aug2026.pdf \
  --private-key-file=sa-key.json --duration=1h
```

🎯 Scenario-Based Interview Questions

Q: A partner company needs to download one specific invoice PDF from your private bucket for exactly the next 30 minutes, and must not be able to see or download anything else. How do you provide access on GCP? Generate a Signed URL scoped to that single object, with the GET method only, an expiry of 30 minutes, and ideally restricted to HTTPS. Share only the resulting URL — no IAM role or Google identity is granted to the partner at all, they cannot list other objects in the bucket (the signature is scoped to the one object path, not the bucket), and after 30 minutes the URL simply stops working with no manual cleanup required. This is functionally identical to an AWS S3 pre-signed URL or an Azure Blob SAS token.

GCSA

CHAPTER 11 · OBJECT STORAGE

Object Storage — Cloud Storage Advanced

Partitioning, gcloud storage transfer, replication, lifecycle, KMS.

Beyond basic upload/download, production use of Cloud Storage requires understanding how data is partitioned for performance, how to move data at scale, how replication protects against regional failure, how lifecycle policies automate cost optimisation, and how encryption and static website hosting round out the platform's capabilities.

11.1 Data Partitioning

📝 Theory & Key Points

- GCS automatically shards and load-balances traffic across its distributed storage system based on the object's name/key.
- Historically, sequential naming patterns (e.g. timestamps as a prefix: 2026-08-16-log1.txt) could create a 'hot' key-range bottleneck under very high, sudden request rates — GCS's autoscaling of request capacity has significantly reduced this concern compared to a decade ago, but Google still recommends avoiding sequential prefixes for extremely high-throughput workloads.
- Best practice for extremely high-throughput workloads: add a random or hashed prefix at the start of the object name to spread load evenly, the same principle used for AWS S3 key-naming.

11.2 Data Copying and Moving

BASH

```
# gcloud storage — the modern high-performance CLI for bulk copy/move (parallelized automatically)

# Copy an entire local folder to a bucket recursively
gcloud storage cp -r ./localfolder gs://devopslab-uploads-2026/

# Copy objects between two buckets directly (server-side, no local download)
gcloud storage cp -r gs://source-bucket/data gs://dest-bucket/data

# Sync a local folder with a bucket (mirrors changes, deletes extras)
gcloud storage rsync ./website gs://devopslab-static-site --delete-unmatched-destination-objects

# For massive data migrations (petabyte scale, from on-prem or another cloud):
# Storage Transfer Service — a managed, scheduled, resumable transfer job
gcloud transfer jobs create s3://source-aws-bucket gs://devopslab-uploads-2026 \
  --source-creds-file=aws-creds.json
```

11.3 Cross-Region / Same-Region Replication

LOCATION TYPE	REPLICATION	PROTECTS AGAINST	NOTES
Region	Data replicated redundantly within one region across multiple zones	Zone-level failure	Lowest latency to compute in that region
Dual-region	Data synchronously OR asynchronously (Turbo Replication) replicated between two specific regions you choose	Full regional disaster in either of the two regions	Turbo Replication offers a 15-minute RPO SLA
Multi-region	Data replicated across a large geographic area (e.g. all 'us' regions)	Regional disaster anywhere in that geography	Highest availability, best for globally-accessed public content

11.4 Lifecycle Management

📝 Theory & Key Points

- An **Object Lifecycle Management** policy is a set of rules that automatically transitions objects between storage classes or deletes them, based on age, number of newer versions, or other conditions — fully automated cost optimisation.
- Common pattern: Standard for 30 days → auto-move to Nearline → auto-move to Coldline after 90 days → auto-move to Archive after 365 days → auto-delete after 7 years for compliance retention, then cleanup.

JSON

```
{
  "rule": [
    {
      "action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
      "condition": {"age": 30, "matchesPrefix": ["logs/"]}
    },
    {
      "action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
      "condition": {"age": 365, "matchesPrefix": ["logs/"]}
    },
    {
      "action": {"type": "Delete"},
      "condition": {"age": 2555, "matchesPrefix": ["logs/"]}
    }
  ]
}
```

11.5 Security & Encryption

📝 Theory & Key Points

- All data in Cloud Storage is **encrypted at rest by default** using AES-256 — this cannot be disabled and requires no configuration.
- By default, Google manages the encryption keys (**Google-managed encryption keys**). For stricter compliance, you can supply **Customer-Managed Encryption Keys (CMEK)** via Cloud KMS, giving you rotation and revocation control, or even **Customer-Supplied Encryption Keys (CSEK)** where you provide the raw key on every request.
- Data in transit is protected via HTTPS/TLS by default for all API access.

11.6 Cloud KMS

📝 Theory & Key Points

- **Cloud Key Management Service (KMS)** is GCP's centralised cryptographic key management service — the equivalent of AWS KMS. It manages symmetric and asymmetric keys, backed optionally by FIPS 140-2 validated HSMs (Cloud HSM) or even an external key manager (Cloud EKM) for full key sovereignty.
- Secrets (as opposed to encryption keys) — passwords, API tokens, connection strings — are more typically stored in **Secret Manager**, GCP's dedicated secrets service, which itself uses Cloud KMS under the hood for envelope encryption.
- Applications retrieve secrets via a **Service Account's** IAM permissions (no embedded credentials at all) rather than hardcoding a key or secret in code.

11.7 Static Website Hosting with Cloud Storage

📝 Theory & Key Points

- A bucket can be configured to serve as a **static website** by setting a main page suffix (e.g. index.html) and a 404 error page, then making the bucket's objects publicly readable — ideal for hosting a React/Vue SPA or a simple static site cheaply.
- Typically paired with **Cloud CDN** (fronted by a Global External HTTP(S) Load Balancer with the bucket as a backend) for a custom domain, edge caching, and global low-latency delivery — this is the standard, recommended production pattern rather than serving directly from the bucket's default domain.

BASH

```
# Make the bucket's objects publicly readable and set it up for website hosting
gcloud storage buckets add-iam-policy-binding gs://devopslab-static-site \
  --member=allUsers --role=roles/storage.objectViewer

gcloud storage buckets update gs://devopslab-static-site \
  --web-main-page-suffix=index.html --web-error-page=404.html

# Upload the site content
gcloud storage cp -r ./dist/* gs://devopslab-static-site/
```

11.8 Events Configuration on Buckets

📝 Theory & Key Points

- GCS integrates with **Pub/Sub Notifications** to publish an event (object finalized/created, deleted, archived, metadata updated) to a Pub/Sub topic the instant something changes in a bucket — GCP's equivalent of Azure Event Grid's Blob events or S3 Event Notifications.
- Common pattern: an uploaded image publishes a Pub/Sub message → a Cloud Function subscribed to that topic automatically generates a thumbnail and writes it back to a different bucket — a fully event-driven, serverless image-processing pipeline.
- **Eventarc** is GCP's more modern, unified eventing layer, letting you route GCS (and 100+ other GCP source) events directly to Cloud Run, Cloud Functions (2nd gen), or GKE with standardised CloudEvents formatting.

🎯 Scenario-Based Interview Questions

Q: Your company must retain financial transaction logs for 7 years for compliance, but only the last 30 days are ever actually queried, and cost is a major concern. Design a storage strategy on GCP. Store logs in a Cloud Storage bucket starting in the Standard class for fast access during the first 30 days when they're actively queried. Configure an Object Lifecycle Management policy to automatically transition objects to Nearline after 30 days (cheaper storage, occasional access), to Coldline after ~90 days, and to Archive after 365 days (cheapest storage — and unlike some competitors, GCS Archive retrieval is still near-instant, no multi-hour rehydration wait), then automatically delete at the 7-year (2555 day) mark once the compliance window closes. Choose a dual-region or multi-region bucket for the regulatory requirement of surviving a regional disaster, and enable Bucket Lock (GCS's WORM — Write Once Read Many retention policy) if the compliance regime requires tamper-proof retention, preventing even a project owner from deleting logs before the retention period expires.

GCSX

CHAPTER 12 · OBJECT STORAGE

Object Storage — Cross-Project Cloud Storage Access

Cross-project Cloud Storage access and VPC Service Controls.

Real-world architectures frequently need one team's application (in Project A) to read or write data in another team's bucket (in Project B) — for example, a shared data lake consumed by multiple analytics teams. GCP provides several controlled mechanisms for this cross-project access, built entirely on Cloud IAM since GCS permissions are IAM-native.

12.1 Enabling Cross-Project Access for Cloud Storage

MECHANISM	HOW IT WORKS	BEST FOR
Signed URL	Time-boxed, permission-scoped URL generated by the resource owner and handed to the consumer	One-off or external-partner access without granting a persistent identity any IAM role
Cross-Project IAM Binding	Grant a role (e.g. Storage Object Viewer) directly on the bucket to a user, group, or Service Account that belongs to a different project	Ongoing, auditable access between teams within the same organisation
VPC Service Controls	Define a security perimeter around one or more projects' GCS (and other) resources, blocking data exfiltration even by an identity with valid IAM permissions if they're outside the perimeter's allowed context	Highest-security, compliance-sensitive cross-project access, defense against credential theft
Private Google Access / Private Service Connect	Ensures traffic from a VPC to Google APIs (including GCS) stays on Google's private network rather than the public internet, even without external IPs	Locking network paths down for compliance, while still using standard IAM for authorization

BASH

```bash
# Grant a Service Account from another project read access via IAM
gcloud storage buckets add-iam-policy-binding gs://shared-data-lake \
  --member="serviceAccount:analytics-job@project-b.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"

# Restrict a bucket's access to only be usable from within a VPC Service Controls perimeter
gcloud access-context-manager perimeters update analytics_perimeter \
  --add-resources=projects/123456789 \
  --policy=my-policy-id
```

🎯 Scenario-Based Interview Questions

Q: Team A owns a bucket with sensitive customer data. Team B (a different GCP project, same company) needs read-only access for a nightly analytics job, and security requires that even a leaked/stolen service account key must not be usable to exfiltrate data from an untrusted network. What do you configure? Grant Team B's analytics job Service Account the 'Storage Object Viewer' IAM role scoped only to the specific bucket it needs — satisfying least privilege for normal operation. Then wrap both Team A's and Team B's projects inside a shared VPC Service Controls perimeter, which enforces that GCS API calls against that bucket are only honoured when they originate from within the trusted perimeter's context (e.g. from an authorized VPC or IP range) — even a correctly-authenticated, correctly-authorized request using a stolen service account key would be blocked if it comes from outside the perimeter (e.g. an attacker's home network), because VPC Service Controls enforces a network-context boundary independently of and on top of IAM.

IAM

CHAPTER 13 · IDENTITY & ACCESS

Identity & Access — Cloud IAM

Cloud IAM — 2-Step Verification, roles, service accounts, auditing.

Cloud IAM (Identity and Access Management) is GCP's system for controlling who (identity) can do what (role) on which resource (scope). Every single API call on GCP — creating a VM, reading a bucket object, deploying a Cloud Function — is authorized through IAM, making it the security backbone of the entire platform.

💡 Layman Explanation

IAM is like the building's central security desk. Every person (user) and every robot courier (application/service account) must badge in here first. The security desk doesn't just check identity — it also knows exactly which floors and rooms (GCP resources) each badge is allowed to enter, based on their assigned role, and at GCP that badge-and-permissions system inherits down through an org chart: Organization → Folder → Project → Resource.

13.1 Root Account vs. IAM User

📝 Theory & Key Points

- GCP has no single 'root account'. Resources are organised in a hierarchy: **Organization** (the company) → **Folders** (e.g. departments/teams) → **Projects** (the actual container for resources like VMs and buckets) → individual **Resources**.
- The **Organization Administrator** role has the broadest tenant-wide administrative rights at the top of the hierarchy; a **Project Owner** has the broadest rights within a single project.
- Best practice: never use an Organization Admin or Project Owner account for daily work. Create individual named user accounts (or better, Groups) with least-privilege predefined roles, and reserve Owner access for a small number of break-glass administrators only.
- IAM policy bindings are **additive and inherit downward** — a role granted at the Organization level automatically applies to every Folder, Project, and Resource beneath it, which is powerful but requires care to avoid unintentional over-permissioning.

13.2 Multi-Factor Authentication (2-Step Verification)

📝 Theory & Key Points

- Google calls MFA **2-Step Verification (2SV)** — requiring a second proof of identity beyond a password, typically via Google Prompt, a security key (Titan/FIDO2), an authenticator app, or SMS.
- For an organisation using Cloud Identity or Google Workspace, 2SV enforcement is managed centrally by the Workspace/Cloud Identity admin, and can be mandated organisation-wide.
- **Context-Aware Access** (part of BeyondCorp Enterprise) goes further than simple MFA — it can require a device to be company-managed, healthy, and coming from an approved network/location before granting access to sensitive resources, regardless of whether the password and 2SV succeeded.

13.3 IAM Password Policies

📝 Theory & Key Points

- Password complexity, length, and expiration policies for Google/Cloud Identity accounts are configured centrally in the Google Admin Console (Workspace/Cloud Identity), not per-project.
- Google increasingly recommends passwordless approaches (Security Keys, Passkeys) over frequent forced password rotation, aligned with modern NIST guidance that shows frequent rotation often leads to weaker, more predictable passwords.
- For Compute Engine VM OS-level accounts (not GCP console access), password policy is a matter for the guest OS itself (e.g. PAM settings on Linux, Group Policy on Windows) — entirely separate from Cloud IAM.

13.4 Keys Configuration and Different Access Approaches

IDENTITY TYPE	USED BY	CREDENTIAL
Google Account (User)	A human logging into the Cloud Console/gcloud	Password + 2-Step Verification
Service Account	An application, VM, or CI/CD pipeline needing to authenticate	Attached automatically to a resource (preferred), or a downloadable JSON key file (higher risk, avoid where possible)
Workload Identity Federation	External workloads (GitHub Actions, AWS, on-prem) needing to authenticate without any long-lived GCP credential	Short-lived tokens exchanged via a trusted external identity provider — no key file at all
Attached Service Account	A single Compute Engine VM, Cloud Function, or GKE workload needing to call other GCP APIs	No credential file at all — retrieved automatically and rotated by Google via the metadata server

⚠️ Important Warning

- Attached Service Accounts (or Workload Identity Federation for external systems) are strongly preferred over downloaded Service Account JSON key files wherever possible — there is no long-lived secret to leak, rotate, or expire.
- Never commit a Service Account JSON key file to source control. If one is ever exposed, treat it as an active security incident: disable/delete the key immediately in IAM and audit Cloud Audit Logs for any activity performed with it.

13.5 Users, Groups, and Role Concepts

📝 Theory & Key Points

- IAM grants access by binding a **Role** (a collection of permissions) to a **Principal** (user, group, service account, or domain) at a specific **Resource scope** (Organization → Folder → Project → Resource).
- Three role categories exist: **Primitive Roles** (Owner, Editor, Viewer — broad, legacy, generally discouraged in production for being too coarse-grained), **Predefined Roles** (hundreds of Google-curated, service-specific roles like Storage Object Viewer or Compute Instance Admin, offering good granularity out of the box), and **Custom Roles** (you assemble an exact permission set yourself).
- Always bind roles to **Groups**, not individual users — when someone joins or leaves a team, you add/remove their group membership once instead of hunting down dozens of individual resource-level bindings.
- **IAM Conditions** let you add context-aware constraints to a binding — e.g. a role only takes effect before a certain expiry date/time, or only for resources matching a certain name prefix — useful for temporary access grants.

```text
Organization  (e.g. "example.com")
    |  a role assigned here applies to EVERYTHING below
    ▼
Folder  (e.g. "Production")
    |
    ▼
Project  (e.g. "my-devops-lab-project")
    |  <- most role bindings happen at THIS level in practice
    ▼
Resource  (e.g. one specific VM, one Cloud Storage bucket)
    |  most granular — narrowest possible blast radius
    ▼
Effective Access = union of every role bound at every level above
```

Fig 5: GCP Resource Hierarchy — IAM Policies Inherit Downward

13.6 Creating Customer-Managed (Custom) Policies

BASH

```
# Create a Google Group for a team (managed via Google Admin Console / Cloud Identity)
gcloud identity groups create devops-engineers@example.com \
  --organization="example.com" --display-name="DevOps Engineers"

# Bind the predefined 'Compute Instance Admin' role to that group, scoped to one
project
gcloud projects add-iam-policy-binding my-devops-lab-project \
  --member="group:devops-engineers@example.com" \
  --role="roles/compute.instanceAdmin.v1"

# Create a custom role: allow starting/restarting VMs but NOT deleting them
gcloud iam roles create vmOperatorNoDelete --project=my-devops-lab-project \
  --title="VM Operator - No Delete" \
  --permissions=compute.instances.start,compute.instances.reset,compute.instances.get
\
  --stage=GA

# Create a Service Account and attach it to a VM, no key file needed
gcloud iam service-accounts create web-tier-sa --display-name="Web Tier Service
Account"
gcloud compute instances set-service-account vm-app01 --zone=asia-south1-a \
  --service-account=web-tier-sa@my-devops-lab-project.iam.gserviceaccount.com
```

13.7 Auditing User Activity

📝 Theory & Key Points

- **Cloud Audit Logs** automatically and continuously record every administrative and data-access operation across nearly all GCP services — who did what, when, and from which IP — with **Admin Activity logs** retained 400 days by default (and always-on, cannot be disabled).
- **Data Access logs** (off by default for most services due to volume/cost, except for a few sensitive services where they're always on) additionally record read/write operations on the data itself, not just configuration changes.
- **Policy Analyzer** and **IAM Recommender** proactively identify overly broad IAM grants (e.g. a Service Account with unused Owner-level access) and suggest tightening them to actual observed usage.
- **Security Command Center** and **Chronicle** (GCP's SIEM) can correlate audit logs to detect anomalous activity, such as impossible-travel logins or privilege escalation attempts.

🎯 Scenario-Based Interview Questions

Q: A contractor needs to deploy and manage VMs in exactly one project for a 3-month engagement, but must never be able to view billing/organization-level settings, and must never be able to grant access to anyone else. What role and scope do you assign on GCP? Grant a predefined role such as **Compute Admin** (roles/compute.admin) — not Owner — bound at the Project level only, not the Folder or Organization. Compute Admin allows full management of Compute Engine resources (create/modify/delete VMs, disks, images) within scope, but has no ability to modify IAM policy bindings (that needs roles/resourcemanager.projectIamAdmin or Owner) and no visibility into the organization's billing account. Use an IAM Condition to set an expiry timestamp 3 months out on the binding itself, so access automatically lapses without anyone needing to remember to revoke it manually.

Q: You're auditing an incident where a production VM was deleted without authorisation. How do you trace what happened using GCP's native tools? Query Cloud Audit Logs (via Cloud Console → Logging → Logs Explorer, or the `gcloud logging read` command) filtered by the resource name of the deleted VM and the method name 'v1.compute.instances.delete' — the Admin Activity log entry reveals exactly which identity (user or service account) performed the delete, the timestamp, and the caller's IP address. Cross-reference that identity against Cloud Identity sign-in activity around that timestamp to confirm it was a legitimate authenticated session with 2-Step Verification, rather than a compromised credential. If access should never have been possible, review the IAM policy bindings at that project (and any inherited from its Folder/Organization) to find and correct the over-permissioning that allowed it.

SQL

CHAPTER 14 · DATABASES

Databases — Cloud SQL

Cloud SQL — PaaS vs IaaS, Cloud SQL Auth Proxy, Memorystore.

Cloud SQL is GCP's fully managed relational database service, supporting MySQL, PostgreSQL, and SQL Server engines. It removes the operational burden of patching, backups, and high-availability configuration — you focus purely on schema design and queries, while Google manages the underlying VM, storage, and replication.

14.1 Launching a Cloud SQL Instance

BASH

```
# Create a PostgreSQL Cloud SQL instance
gcloud sql instances create sql-devopslab \
  --database-version=POSTGRES_15 \
  --tier=db-custom-2-7680 \
  --region=asia-south1 \
  --root-password="P@sswOrd12345!" \
  --availability-type=ZONAL

# Authorize your current IP through the instance's firewall
gcloud sql instances patch sql-devopslab \
  --authorized-networks=203.0.113.10/32

# Create the actual database on that instance
gcloud sql databases create appdb --instance=sql-devopslab
```

14.2 Difference Between DB on Compute Engine vs Managed Cloud SQL

ASPECT	DATABASE ON COMPUTE ENGINE (IAAS)	CLOUD SQL (MANAGED/PAAS)
OS/Patching	You manage the Linux/Windows OS + database engine patching	Fully managed by Google, including OS and engine patches
Backups	You configure and manage backup jobs	Automated daily backups + point-in-time recovery built in
High Availability	You build it yourself (replication, failover tooling)	Built-in regional HA option with automatic failover to a standby in a different zone
Control	Full OS/instance-level access, any database extension, custom agent jobs	No OS access; a curated (though large) set of supported extensions/flags
Best for	Legacy apps needing full OS-level control or unsupported extensions	New/cloud-native apps wanting minimal operational overhead
Scaling	Manual — resize the VM, reconfigure storage	Change machine tier or enable storage auto-increase with a single command, often with minimal downtime

14.3 Proxy Configuration Process

📝 Theory & Key Points

- The **Cloud SQL Auth Proxy** is GCP's signature connectivity tool — a lightweight local process that creates a secure, IAM-authenticated tunnel between your application (running anywhere: Compute Engine, GKE, Cloud Run, or your laptop) and your Cloud SQL instance, without needing to whitelist IP addresses or manage SSL certificates manually.
- The proxy authenticates using the caller's IAM identity/Service Account credentials and automatically encrypts the connection with TLS, even if the application code itself just connects to 'localhost' on the proxy's local port.
- This is the Google-recommended, most secure connection method — far preferred over authorizing individual public IP addresses on the instance's firewall, which the proxy makes largely unnecessary.

```bash
# Download and run the Cloud SQL Auth Proxy locally
curl -o cloud-sql-proxy https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/v2.11.0/cloud-sql-proxy.linux.amd64
chmod +x cloud-sql-proxy

./cloud-sql-proxy my-devops-lab-project:asia-south1:sql-devopslab --port=5432

# In another terminal, connect exactly as if the DB were local
psql "host=127.0.0.1 port=5432 sslmode=disable dbname=appdb user=postgres"
```

14.4 Memorystore (ElastiCache Equivalent)

📝 Theory & Key Points

- **Memorystore** is GCP's fully managed, in-memory data store, available for both **Redis** and **Memcached** engines — the direct equivalent of AWS ElastiCache — used to reduce database load and slash read latency from milliseconds to microseconds.
- Common patterns: caching frequent, expensive SQL query results; storing user session state for a horizontally-scaled web tier; implementing a distributed lock or rate limiter; acting as a lightweight pub/sub bus for real-time features.
- Memorystore for Redis tiers range from Basic (single node, no SLA, dev/test) to Standard (a replicated pair with automatic failover) to Memorystore for Redis Cluster (horizontal sharding for very high throughput).

```bash
# Create a Standard-tier Memorystore Redis instance
gcloud redis instances create redis-devopslab \
  --size=5 --region=asia-south1 --tier=standard --redis-version=redis_7_0

# Get the connection host/port needed by the application
gcloud redis instances describe redis-devopslab --region=asia-south1
```

🎯 Scenario-Based Interview Questions

Q: A read-heavy product catalog page hits the Cloud SQL database on every page load, and the DB is now the bottleneck at peak traffic. The catalog data changes only a few times a day. What's the fix? Introduce Memorystore for Redis as a read-through cache in front of the database: on a page load, the application first checks Redis for the catalog data; on a cache miss it queries Cloud SQL once, then writes the result into Redis with a TTL (e.g. 15 minutes, or invalidated explicitly whenever the catalog is updated). Since the catalog changes only a few times daily, the cache hit rate will be extremely high, cutting the vast majority of read traffic away from the database entirely and moving it to Redis's sub-millisecond in-memory lookups — Cloud SQL is then free to handle writes and less-cacheable queries, and can potentially be sized down.

SQLOP

CHAPTER 15 · DATABASES

Databases — Cloud SQL Operations

Connecting to Cloud SQL, running queries, and read replicas.

Day-to-day work with Cloud SQL involves connecting from application servers and local developer machines, running everyday DDL/DML, and configuring read replicas for reporting workloads or read-scale needs — all without ever touching the underlying OS.

15.1 Connecting from Compute Engine

📝 Theory & Key Points

- From a Compute Engine VM in the same project, the simplest and most secure method is the **Cloud SQL Auth Proxy** running as a sidecar process on the VM, authenticated via the VM's attached Service Account — no passwords or IP allowlisting required.
- Alternatively, **Private IP** connectivity lets the Cloud SQL instance appear directly inside your VPC via VPC Peering (Private Service Access), letting the VM connect over an internal IP with no proxy or public exposure at all — the recommended pattern for production VPC-native architectures.

BASH

```
# Option 1: Cloud SQL Auth Proxy running on the VM itself
./cloud-sql-proxy my-devops-lab-project:asia-south1:sql-devopslab --port=5432 &
psql "host=127.0.0.1 port=5432 dbname=appdb user=postgres"

# Option 2: Private IP — connect directly if Private Service Access is configured
gcloud sql instances patch sql-devopslab --network=vpc-prod --no-assign-ip
psql "host=<private-ip> dbname=appdb user=postgres"
```

15.2 Connecting from a Local Machine via Workbench

📝 Theory & Key Points

- From a laptop, run the Cloud SQL Auth Proxy locally (authenticated via `gcloud auth application-default login` or a downloaded Service Account key) — this is strongly preferred over adding the laptop's public IP to the instance's authorized networks list.
- Connect using MySQL Workbench, pgAdmin, or DBeaver pointed at 127.0.0.1 on the proxy's local port, exactly as if the database were running locally.
- For production, prefer **IAM Database Authentication** (supported for MySQL and PostgreSQL) over static database passwords — it lets a user or Service Account authenticate to the database using their GCP IAM identity, enabling MFA and centralised access revocation through the same identity system as everything else in GCP.

15.3 Creating DBs, Tables, and Running Queries

SQL

```sql
-- Create a table
CREATE TABLE orders (
    order_id     SERIAL PRIMARY KEY,
    customer_id  INT NOT NULL,
    order_date   TIMESTAMPTZ DEFAULT now(),
    status       VARCHAR(20) DEFAULT 'PENDING',
    total_amount NUMERIC(10,2) NOT NULL
);

-- Insert a row
INSERT INTO orders (customer_id, total_amount) VALUES (1042, 249.99);

-- Query with an index-friendly filter
SELECT order_id, total_amount, status
FROM orders
WHERE customer_id = 1042 AND order_date > now() - interval '30 days';

-- Create an index to speed up the above query
CREATE INDEX idx_orders_customer_date ON orders (customer_id, order_date);
```

15.4 Configuring Read Replicas

📝 Theory & Key Points

- Cloud SQL supports up to several **Read Replicas** — automatically kept in near-real-time sync via the database engine's native replication — that reporting/analytics queries can be routed to, so they never compete with production write traffic on the primary.
- Read Replicas can be created in the same region (for read scaling) or a different region (cross-region replicas, additionally useful as a disaster-recovery target that can be manually promoted to a standalone primary during a regional outage).
- Applications simply point their read-only queries at the replica's separate connection endpoint/IP — no special connection-string flag is needed the way Azure SQL uses `ApplicationIntent=ReadOnly`, since a GCP read replica is a distinct addressable instance.

BASH

```
# Create a read replica of an existing Cloud SQL instance
gcloud sql instances create sql-devopslab-replica1 \
  --master-instance-name=sql-devopslab \
  --tier=db-custom-2-7680 --region=asia-south1

# Promote a replica to a standalone primary (e.g. during a DR failover)
gcloud sql instances promote-replica sql-devopslab-replica1
```

🎯 Scenario-Based Interview Questions

Q: The reporting team's nightly dashboard queries are so heavy they're causing timeouts for real production checkout transactions on the same Cloud SQL database. Your fix must not require standing up a separate database or ETL pipeline. Create a Cloud SQL Read Replica of the existing production instance and point the reporting team's connection string at the replica's own IP/endpoint instead of the primary. Their heavy analytical queries are then automatically isolated onto a database engine-replicated copy that never touches the primary's write path, without provisioning any new database engine, running an ETL job, or changing the reporting queries themselves, since the replica is schema-identical and stays near-real-time in sync via native replication.

APP

CHAPTER 16 · SERVERLESS

Serverless — Cloud Functions & App Engine

App Engine, Cloud Functions, and Cloud Run — serverless compute options.

Serverless computing lets you run code without provisioning or managing any servers — the cloud provider handles scaling, patching, and infrastructure entirely, and you often pay only for actual execution time. GCP offers two flagship serverless compute services with different philosophies: App Engine (a fully managed application platform) and Cloud Functions (granular, event-driven functions) — alongside Cloud Run, which sits between them, running any containerised workload serverlessly.

💡 Layman Explanation

Running your own VM is like owning a restaurant — you handle the building, the ovens, the electricity, staffing shifts, even when no customers show up. Serverless is like a cloud kitchen: you just supply the recipe (your code); the kitchen appears the instant an order (a request/event) comes in, cooks it, and disappears — you're billed per dish made, not per hour the kitchen sits idle.

16.1 Overview of Cloud Functions / App Engine

📝 Theory & Key Points

- **App Engine** is GCP's original fully managed PaaS for hosting complete web apps and APIs in numerous languages (Python, Java, Node.js, Go, PHP, Ruby), without managing the underlying VM/OS at all — you deploy source code and App Engine builds, deploys, and scales it.
- App Engine has two environments: **Standard** (sandboxed, scales to zero, fastest cold starts, some language-runtime restrictions) and **Flexible** (runs in Docker containers on Compute Engine VMs under the hood, more customisable, does not scale to zero).
- **Cloud Functions** is GCP's Function-as-a-Service (FaaS) offering — the equivalent of AWS Lambda — running small, single-purpose pieces of code in response to a **trigger** (HTTP request, Pub/Sub message, Cloud Storage event, Firestore change).
- **Cloud Run** (worth knowing alongside these) runs any stateless container serverlessly, scaling to zero, and is increasingly the default recommendation for new serverless workloads at Google since it combines container flexibility with true serverless economics.

16.2 Configuration with Different Services

SERVICE	DEPLOYMENT UNIT	SCALE-TO-ZERO	BEST FOR
App Engine Standard	Source code, managed runtime	Yes	Traditional web apps/APIs wanting zero infrastructure management
App Engine Flexible	Docker container on a managed VM	No (minimum 1 instance)	Apps needing custom runtimes/libraries not in the Standard sandbox
Cloud Functions (1st/2nd gen)	A single function's source code	Yes	Small, event-driven, single-purpose pieces of logic
Cloud Run	Any container image	Yes	General-purpose containerised services, APIs, and even Cloud Functions 2nd gen (built on Cloud Run under the hood)

16.3 Creation and Automation Using Cloud Functions

📝 Theory & Key Points

- On the default pricing model, you pay only per invocation + execution time (GB-seconds and GHz-seconds), and Google scales instance count from zero to thousands automatically — the purest serverless economics.
- **Cloud Functions 2nd gen** is built on top of Cloud Run and Eventarc, giving longer request timeouts (up to 60 minutes vs 9 minutes on 1st gen), larger instance sizes, concurrency (multiple requests per instance), and a much broader set of native event sources via Eventarc.
- **Minimum instances** can be configured to keep a warm pool ready, eliminating cold-start latency for latency-sensitive endpoints — analogous to Azure Functions' Premium plan pre-warmed instances.

BASH

```
# Deploy an HTTP-triggered Cloud Function (2nd gen, Python)
gcloud functions deploy process-order \
  --gen2 --runtime=python312 --region=asia-south1 \
  --source=. --entry-point=process_order \
  --trigger-http --allow-unauthenticated

# Deploy a Pub/Sub-triggered function (event-driven, no HTTP)
gcloud functions deploy on-new-upload \
  --gen2 --runtime=python312 --region=asia-south1 \
  --source=. --entry-point=handle_upload \
  --trigger-topic=new-uploads-topic

# Deploy a full web app to App Engine Standard
# (app.yaml in the project root defines runtime + scaling)
gcloud app deploy app.yaml
```

16.4 General Configuration and Environment Setup

📝 Theory & Key Points

- **Environment Variables** are configured outside the code, injected at runtime — never hardcode secrets in the codebase.
- For anything sensitive, reference a **Secret Manager** secret directly as a mounted volume or environment variable, combined with the function/app's Service Account IAM permissions for credential-free retrieval.
- **Traffic Splitting** (App Engine) or **Revisions with traffic percentages** (Cloud Run) let you deploy a new version alongside the old one and gradually shift a percentage of traffic — a built-in canary deployment mechanism, with instant rollback by shifting traffic back.

16.5 Layers Equivalent and Importance

📝 Theory & Key Points

- GCP has no direct, literal equivalent of AWS Lambda's 'Layers' feature for Cloud Functions. The closest analogs are: (1) a shared requirements.txt/package.json dependency file referencing a private package published to **Artifact Registry**, reused across multiple functions' source, or (2) for Cloud Run / Cloud Functions 2nd gen, building on top of a shared custom **base container image** stored in Artifact Registry that bundles common dependencies.
- The core benefit is the same as Lambda Layers: avoid duplicating large dependencies across every function's deployment package, speed up deployment, and centralise version updates to shared code.
- For genuinely large shared libraries, packaging them as a private Python/Node package in Artifact Registry (GCP's package + container registry) and simply listing it as a dependency is the most idiomatic GCP-native pattern.

🎯 Scenario-Based Interview Questions

Q: You're deploying a new version of a critical API and management is nervous about downtime or a bad release. What GCP feature directly addresses this, and how does it work?
Use Cloud Run's traffic-splitting between revisions (or App Engine's traffic splitting between versions, if using App Engine). Deploy the new version as a new Revision — it doesn't receive any traffic automatically — then gradually shift a small percentage (e.g. 5%, then 25%, then 100%) of live traffic to it while monitoring error rates and latency via Cloud Monitoring. If anything looks wrong at any stage, you shift traffic back to the previous, known-good revision instantly with a single command, achieving a safe canary rollout and near-instant rollback without ever taking the service offline or requiring a brand-new deployment to revert.

APPI

CHAPTER 17 · SERVERLESS

Serverless — Cloud Functions Integrations & Limitations

Function limitations and integrations — Cloud SQL, Firestore, API Gateway.

Serverless compute rarely stands alone — Cloud Functions is most powerful when wired directly into other managed GCP services, letting you build event-driven pipelines with almost no custom plumbing code. This section covers connecting Functions to Cloud SQL, Firestore, and API Gateway, plus the practical limitations to plan around.

17.1 Limitations of Cloud Functions

📝 Theory & Key Points

- **Cold starts:** a function that hasn't run recently must 'spin up' a new instance on the next request, adding latency (typically hundreds of ms to a few seconds) — mitigated by configuring minimum instances to keep a warm pool.
- **Execution time limits:** 1st gen functions default to a 9-minute maximum timeout; 2nd gen (built on Cloud Run) extends this to 60 minutes for HTTP-triggered functions — genuinely long-running workflows should use Workflows or Cloud Composer for orchestration instead.
- **Stateless by design:** a function instance can be recycled at any time, so it must not rely on in-memory state persisting between invocations — any state needed must live in an external store (Cloud SQL, Firestore, Memorystore, Cloud Storage).
- **Concurrency differences:** 1st gen functions handle exactly one request per instance at a time; 2nd gen (Cloud Run-based) functions can handle many concurrent requests per instance, which changes both cost and scaling characteristics significantly.

17.2 Cloud Functions to Cloud SQL Connection

BASH

```
# Deploy with the Cloud SQL instance connection configured directly
gcloud functions deploy insert-order \
  --gen2 --runtime=python312 --region=asia-south1 \
  --source=. --entry-point=insert_order --trigger-http \
  --set-env-vars=INSTANCE_CONNECTION_NAME=my-devops-lab-project:asia-south1:sql-
devopslab \
  --set-secrets=DB_PASSWORD=db-password:latest
```

PYTHON

```
# Python function using the Cloud SQL Python Connector (pseudocode)
from google.cloud.sql.connector import Connector
import sqlalchemy, os

connector = Connector()
def getconn():
    return connector.connect(
        os.environ["INSTANCE_CONNECTION_NAME"], "pg8000",
        user="postgres", password=os.environ["DB_PASSWORD"], db="appdb")

pool = sqlalchemy.create_engine("postgresql+pg8000://", creator=getconn)
```

17.3 Cloud Functions to Firestore and API Gateway Connection

📝 Theory & Key Points

- **Firestore Triggers** (via Eventarc for 2nd gen functions) fire automatically whenever documents are created, updated, or deleted in a Firestore database — ideal for real-time reactive pipelines (e.g. re-index a search service the instant a product document changes), with no polling required.
- Firestore is GCP's serverless NoSQL document database — the equivalent of AWS DynamoDB or Azure Cosmos DB — offering real-time listeners, offline sync for mobile/web clients, and strong consistency.
- **API Gateway** (or the more advanced **Apigee** for enterprise API management) sits in front of Cloud Functions/Cloud Run as a managed API gateway, providing a single, versioned, documented, and secured entry point — handling API key/OAuth validation, rate limiting/throttling, request transformation via an OpenAPI spec, while the Function stays focused purely on business logic.

TEXT



Fig 6: Event-Driven Serverless Pipeline — Storage Upload to Notification

🎯 Scenario-Based Interview Questions

Q: You want external partners to call your Cloud Functions-based pricing API, but need to enforce per-partner rate limits, require API keys, and hide the underlying Cloud Function's URL entirely. What do you add in front of the function? Place API Gateway in front of the Cloud Function as the sole public entry point, defined by an OpenAPI specification. Configure API Gateway with an API key requirement (or OAuth) per partner, and use Cloud Endpoints/quota configuration for per-partner rate limiting. Callers only ever see the API Gateway's domain — the actual Cloud Function's default *.cloudfunctions.net URL is never exposed and can be locked down to only accept authenticated calls carrying API Gateway's own service identity, giving both access control and a clean abstraction layer in front of the business logic.

TF

CHAPTER 18 · INFRASTRUCTURE AS CODE

Infrastructure as Code — Terraform with GCP

Terraform with the google provider — state, modules, CI/CD workflow.

Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp that lets you define GCP resources — Compute Engine VMs, VPCs, Cloud Storage buckets, Cloud SQL instances, everything covered in this document — in declarative configuration files, then plan and apply changes in a repeatable, version-controlled, peer-reviewed way, instead of clicking through the Cloud Console.

💡 Layman Explanation

Clicking through the Cloud Console to build infrastructure is like building furniture from memory, one piece at a time, with no instruction manual — hard to repeat exactly, hard to hand off to someone else, and easy to forget a step. Terraform is the IKEA instruction manual: a precise, written blueprint that anyone (or any automated pipeline) can follow to build the exact same result every single time, in any region, as many times as needed.

18.1 Core Terraform Concepts

📝 Theory & Key Points

- A **Provider** (the `google` or `google-beta` provider) is the plugin that translates Terraform's generic language into actual GCP API calls.
- A **Resource** block declares one piece of infrastructure (a VM, a VPC). Terraform computes the difference between your declared configuration and the real, current state of the world, and only changes what's actually different.
- The **State File** (terraform.tfstate) is Terraform's record of what it believes exists and its exact IDs/attributes. For team use, state must be stored remotely — a **Cloud Storage bucket** is the idiomatic GCP backend, which natively supports state locking to prevent concurrent conflicting applies.
- The core workflow is always: **terraform init** (download providers, configure the GCS backend) → **terraform plan** (preview changes, no-op) → **terraform apply** (execute changes) → **terraform destroy** (tear down).

18.2 Example Terraform Configuration — Compute Engine VM + VPC

HCL

```
# providers.tf — configure the GCS remote state backend + the google provider
terraform {
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.40" }
  }
  backend "gcs" {
    bucket = "devopslab-tfstate-bucket"
    prefix = "prod/state"
  }
}

provider "google" {
  project = "my-devops-lab-project"
  region  = "asia-south1"
}

# main.tf — VPC, subnet, firewall rule, and a Compute Engine instance
resource "google_compute_network" "vpc" {
  name                    = "vpc-tf"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "web" {
  name          = "snet-web-tf"
  ip_cidr_range = "10.0.1.0/24"
  region        = "asia-south1"
  network       = google_compute_network.vpc.id
}

resource "google_compute_firewall" "allow_ssh_iap" {
  name          = "allow-iap-ssh-tf"
  network       = google_compute_network.vpc.id
  direction     = "INGRESS"
  source_ranges = ["35.235.240.0/20"]
  allow { protocol = "tcp"; ports = ["22"] }
```

```
}

resource "google_compute_instance" "web01" {
  name         = "vm-tf-web01"
  machine_type = "e2-medium"
  zone         = "asia-south1-a"

  boot_disk {
    initialize_params { image = "debian-cloud/debian-12" }
  }
  network_interface {
    subnetwork = google_compute_subnetwork.web.id
    access_config {}  # gives it an ephemeral external IP
  }
  metadata_startup_script = file("startup.sh")
}
```

BASH

```
# The standard Terraform workflow
terraform init            # download the google provider + configure the GCS backend
terraform fmt              # auto-format the .tf files consistently
terraform validate         # check syntax and internal consistency
terraform plan -out=tfplan # preview exactly what will change
terraform apply tfplan     # execute the plan, create/modify resources
terraform state list       # see everything Terraform is tracking
terraform destroy          # tear everything down (use with extreme caution)
```

18.3 State Management & Team Workflow

📝 Theory & Key Points

- **Remote State with Locking:** storing state in a Cloud Storage bucket automatically provides locking (GCS supports native object generation preconditions Terraform uses for this), preventing two engineers from running `terraform apply` simultaneously and corrupting the state.
- Separate state files per environment (dev/staging/prod), via distinct backend prefixes or entirely separate buckets/projects, keep environments fully isolated so a mistake in dev can never touch prod's state — a common and highly recommended GCP pattern is **one GCP Project per environment**, which also gives natural IAM and billing isolation for free.
- **Modules** let you package reusable, parameterised infrastructure patterns (e.g. a standard '3-tier web app' module) and reuse them across projects instead of copy-pasting HCL.
- Terraform should always run inside a CI/CD pipeline (Cloud Build, GitHub Actions, or similar) for production changes — plan on every Pull Request for visible review, apply only after merge/approval, never applied ad-hoc from an engineer's laptop against production. Using **Workload Identity Federation** lets the CI/CD pipeline authenticate to GCP with no long-lived Service Account key at all.

⚠️ Important Warning

- Never manually edit resources created by Terraform directly in the Cloud Console — this causes 'configuration drift' where the real world no longer matches the state file, leading to unpredictable behaviour on the next apply.
- Never commit the .tfstate file or any .tfvars file containing secrets to Git — state files can contain sensitive data in plaintext (e.g. generated passwords); use a remote GCS backend + Secret Manager-sourced variables instead.

🎯 Scenario-Based Interview Questions

Q: Two engineers on your team both ran 'terraform apply' within a minute of each other against the same production environment, and the state file is now corrupted/inconsistent. How should the team be set up to prevent this, and how do you recover? Prevention: configure a GCS backend for state — Cloud Storage's object generation preconditions give Terraform native locking, meaning a second concurrent apply is automatically blocked until the first one finishes, making this scenario structurally impossible rather than just discouraged by process. Enforce that all applies to production happen only through a CI/CD pipeline (Cloud Build or similar, which naturally serialises runs) rather than from individual laptops. Recovery: use `terraform state list` and `terraform plan` to see what Terraform currently believes exists versus the real GCP

resources, use `terraform state pull` to inspect the raw state, and if genuinely corrupted, restore a previous version of the state object from the bucket's Object Versioning (which is why versioning should always be enabled on the state bucket) before carefully reconciling any resources created during the conflicting applies with `terraform import`.

GCP Cloud & DevOps — Master Study Guide, compiled from the complete GCP Cloud & DevOps Study Notes by Veera Sir, covering Cloud Fundamentals, Compute Engine, Load Balancing, Billing & Monitoring, Autoscaler, Storage Services (Persistent Disk & Filestore), VPC Networking, Object Storage (Cloud Storage), Identity & Access (Cloud IAM), Databases (Cloud SQL), Serverless (App Engine, Cloud Functions, Cloud Run), and Infrastructure as Code (Terraform).

Theory · Diagrams · Commands · Scenario-Based Interview Q&A